A Proactive On-Demand Content Placement Strategy in Edge Intelligent Gateways

Hui Sun, *Member, IEEE*, Yiru Chen, Kewei Sha, *Senior Member, IEEE*, Shaoyuan Huang, Xiaofei Wang, *Senior Member, IEEE*, and Weisong Shi, *Fellow, IEEE*

***Abstract*—Bandwidth-intensive applications transmit large-scale video data in the network. It causes backhaul bottlenecks and affects user experience. Deploying edge cache on an access point (AP) is a popular method to bring content files closer to end-users, but it faces significant challenges, especially in efficiently predicting and satisfying different users' future content requests with limited cache capacity. In this article, we propose an intelligent gateway assisted edge cache deployment strategy (GACD), which jointly considers traffic usage patterns in multiple APs and the impact of new content on the cache performance. In GACD, The cache content placement problem is formulated as a many-to-one bidirectional matching problem with a dynamic quota allocation, aiming to improve cache resource utilization and minimize the average delivery latency. To address this problem, we design a heterogeneous information networks based prediction algorithm to predict end-users' potential preference of new content files. Then, we adapt the seasonal autoregressive integrated moving average model for traffic usage prediction, and propose a many-to-one matching algorithm to achieve dynamic matching quota adjustment and efficient cache content placement. We conduct extensive real-world trace-based experiments to validate the performance of GACD. Compared with six alternative cache strategies, GACD improves the hit rate by 23.9% on average, reduces the average content delivery delay by 19.02%, and increases the accuracy by 31.02% on average.**

***Index Terms*—Cache content placement, edge cache, intelligent gateway, popularity prediction.**

I. Introduction

A REPORT from Cisco [1] estimates that by 2022, the global mobile data traffic will exceed 77 exabytes per month, and data traffic will reach one zettabyte per year with video data accounting for 82% of Internet traffic. Video streaming (e.g., YouTube, Netflix, etc.) has become dominant traffic in the network as multimedia services increase dramatically. The content traffic is overgrowing with the emergence of the bandwidth-intensive applications (e.g., high-definition video applications and interactive games), causing backhaul bottlenecks and poor quality of experience (QoE) [2]. Existing streaming service providers adopt content delivery networks (CDNs) to improve the quality of the end-user experience [3].

Most CDN service providers are classified into two groups- the content owner and the third party. The content owner (e.g., Netflix, Google) have their own CDN infrastructure such as Netflix's Open Connect [4], Google's Global Cache [5]. Thus, end-users can connect directly to their web servers for the required video stream from these CDN server providers. The third-party CDN service providers (e.g., Akamai [6] and Limelight [7]), which are not the content owners, can assist content providers in delivering content to end-users. These content providers can save infrastructure costs but pay the operational and management costs for CDN services providers. Generally, these CDN servers are deployed on geographically distributed backbone nodes on the Internet to redirect user requests to the nearest servers. However, the distance from end-users to their CDN servers is still long [8], which may result in high latency and makes it difficult to provide satisfactory user-perceived QoE [9], especially in large-scale video streaming applications. Recently, CDN providers have been increasingly extended to the network edge [10] to tackle explosive traffic challenges and alleviate the burden on backhaul links. Specifically, CDNs are pushing their content delivery infrastructure closer to the end-user by leveraging storage resources at the edge [11] to achieve fast content delivery and serve the end-users with high-quality content. Moreover, the collaboration between CDNs and Internet Service Providers (ISPs) can improve service quality and reduce service costs (e.g., latency and bandwidth costs for content delivery) [12]. As ISPs manage the entire network and hold network topology and routing information; therefore, CDNs can leverage their routing information to establish a better request redirection mechanism collaborating with ISPs at the edge of the network [13]. Meanwhile, ISPs can benefit from deploying CDN caching servers at the edge of their networks as the total amount of traffic transmitted in the backbone is reduced.

Currently, edge caches [9], [14], [15] provide a promising auxiliary to deliver video content with lower cost and higher quality. According to Cisco [16], the number of global public Wi-Fi hotspots will grow from 169 million in 2018 to 628 million in 2023, and Wi-Fi traffic will account for more than half of total traffic by 2022. The pervasive deployment of Wi-Fi

Manuscript received 24 May 2022; revised 22 January 2023; accepted 21 February 2023. Date of publication 28 February 2023; date of current version 25 May 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 61702001. Recommended for acceptance by J. Wang. *(Corresponding author: Kewei Sha.)*

Hui Sun and Yiru Chen are with the School of Computer and Science, Anhui University, Hefei 230093, China (e-mail: sunhui@ahu.edu.cn; e20201017@stu.ahu.edu.cn).

Kewei Sha is with the Department of Computer Science, University of Houston - Clear Lake (UHCL), Houston, TX 77204 USA (e-mail: sha@uhcl.edu).

Shaoyuan Huang and Xiaofei Wang are with the College of Intelligence and Computing, Tianjin University, Tianjin 300072, China (e-mail: hsy_23@tju.edu.cn; xiaofeiwang@tju.edu.cn).

Weisong Shi is with the Department of Computer and Information Sciences, University of Delaware, Newark, DE 19716 USA (e-mail: weisong@udel.edu).

Digital Object Identifier 10.1109/TPDS.2023.3249797

access points (APs) makes it suitable to cache content at the network edge [17]. These different Wi-Fi APs (e.g., HiWiFi ,[1] Linksys Smart WiFi [2]) are equipped with limited caching, computing, and wireless communication functionalities [18]. They can prefetch popular content files from the cloud through a gateway to provide low-latency services. These gateways bridge the APs and the cloud work as a central coordinator to efficiently manage limited cache resources of APs. This approach has become more promising owing to the emergence of intelligent gateways (e.g., AWS gateway [19], Four-Faith gateway [20], MC-Edge gateway [21]), which have functions such as data collection, protocol conversion, data storage, intelligent computing, data analysis, and decision-making [22]. Several previous studies [22], [23], [24] have applied intelligent gateways in content caching and resources management.

The authors of [24], [25], [26], [27] have exploited cache content placement in wireless networks. Cao et al. [25] design an optimal auction mechanism from the perspective of service providers by considering the costs of content acquisition and delivery. In [27], a reinforcement learning based framework is employed to optimize cooperative caching. These studies assume a known Zipf-like popularity distribution, but they ignore the impact of dynamic end-user requests on content placement. Several recent efforts [15], [28] target to predict dynamic changes in content popularity, but here lack efficient cache content placement strategies. In [29], [30], [31], researchers focus on the content file popularity prediction to determine content placement. However, they rarely consider the uneven distribution of end-users' requests. Furthermore, they do not jointly take into account the amount and the popularity of cache content when designing the cache content placement strategy. Unlike that in the traditional memory cache architecture, caching content files at APs with few user associations in the edge cache architecture can result in high bandwidth overhead and consume storage and computing resources, while it may only bring little cache benefits (e.g., increasing the hit rate). Our design is inspired and built upon the following observations from the analysis of real-world traces from Youku Tudou Inc (Youku). First, at each AP, the amount of both traffic and user associations varies dramatically within a day. Performing proactive content placement during off-peak periods can alleviate backhaul congestion. Second, different APs exhibit different patterns of user associations and network traffic. Dynamic content placement based on these patterns is essential to reduce cache placement cost. Third, random access behaviors of users require to cache highly diverse files at different APs to better satisfy various requirements and minimize the overall delay. Fourth, with the continuous addition of new content, predicting their popularity can optimize cache placement performance. Above findings inspire us to design an intelligent gateway assisted large-scale AP cache deployment (GACD) strategy for two correlated research problems, *i.e.,* how many content files should be placed on each AP, and which content file should be placed on which AP, aiming to both enhance cache resource utilization and minimize the average delivery delay.

In GACD, we first formally model the cache placement problem as a Helper decision problem which is proven to be an NP-hard combinatorial problem. Then, we design a set of novel algorithms, including the unknown domain files popularity prediction algorithm and the many-to-one file placement matching algorithm, which are deployed on an intelligent gateway to manage the cache placement at all connected APs. The contributions of this study are summarized as follows.

▷We have studied a real-world trace, which consists of 53,743 user-AP associations and 155,991 user requests on 41,124 video clips. The observed patterns are used to optimize system design.

▷The proposed GACD strategy and formulation of cache placement problem as a many-to-one bidirectional matching problem with dynamic quota allocation jointly consider traffic patterns and cache file preferences to minimize the average latency. ▷The presented unknown domain files popularity prediction algorithm based on the heterogeneous information network efficiently predicts end-users' preference of new content files. Based on the seasonal autoregressive integrated moving average model for traffic usage prediction, the proposed many-to-one file placement matching algorithm achieves dynamic matching quota adjustment and efficient cache content placement. ▷We perform a comprehensive evaluation on the real-world Youku trace. The results confirm that our method outperforms other strategies in terms of the hit rate, average delay, and accuracy.

The rest of this article is organized as follows. We present our observations of real-world video traffic trace and formulate the cache content placement problem in Section II. Section III elaborates on the design of GACD. Then, we describe the details of the unknown domain files popularity prediction algorithm and the many-to-one file placement matching algorithm in Section IV. In Sections V and VI, we demonstrate the experiment settings and evaluation results, respectively. Related work is discussed in Section VII. Finally, we draw the conclusion in Section VIII.

II. Motivation and Problem Formulation

In this section, we first introduce an edge cache framework based on APs and an intelligent gateway. Second, we study the trace to understand the challenges in edge cache placement and the patterns of user-AP association and user-content request. Based on that, we model the cache content placement problem. The notations and definition used in this article are listed in Table I.

A. System Description

We illustrate a popular edge-caching scenario consisting of a single intelligent gateway, M APs, and K end-users randomly located in the network, see Fig. 1. Let $\mathbf{M} = \{AP_1, AP_2, AP_3, \ldots, AP_M\}$ and $\mathbf{U} = \{u_1, u_2, u_3, \ldots, u_K\}$ denote the set of APs and end-users, respectively. The intelligent gateway connects to the cloud through a high-bandwidth link. The APs connect to the gateway through a bandwidth-constrained backhaul link. End-users access an AP through a bandwidth-constrained front-haul link. Let $\mathbf{F} =$

[1]http://www.hiwifi.com/j2
[2]http://www.linksys.com/en-us/smartwifi

TABLE I
NOTATIONS AND DEFINITION

Notations	Definition
$\mathbf{U}$	Set of end-users, and $\mathbf{U} = \|K\|$
$\mathbf{M}$	Set of APs, and $\mathbf{M} = \|M\|$
$\mathbf{F}$	Set of content files, and $\mathbf{F} = \|F\|$
$\mathbf{S}$	Set of matching quota of each AP
$\mathbf{I}$	Set of end-user request files
$\mathbf{T}$	Set of the future traffic consumption of each AP
$\mathbf{W}$	Set of the traffic weight of each AP
C_{AP}	the cache capacity of the AP
s_f	the size of the file
$r_{k,i}$	the end-user u_k requests file i
$z_{m,f}$	the content file f is placed in the AP_m
$g_{k,m}$	the channel fading coefficient between AP_m and end-user u_k
$h_{k,m}$	the channel gain between AP_m and end-user u_k
$dis_{k,m}$	the Euclidean distanc between AP_m and end-user u_k
$\upsilon_{k,m}$	the transmission rate from AP_m to the end-user u_k
$\tau^1_{\mathrm{k,i}}$	the wireless transmission delay of end-user u_k to obtain file i
$\tau^2_{\mathrm{k,i}}$	the backhaul delay of end-user u_k to obtain file i
$d^m_{k,i}$	the transmission latency for end-user u_k to obtain content file i from its coverage AP_m
$l_{m,k}$	the association between an end-user u_k and AP_m
$p_{m,f}$	the content popularity of the file f in an AP_m area
$\bar{p}_{m,f}$	the content popularity of the known domain file f in an AP_m area
$\widehat{p_{m,f}}$	the content popularity of the unknown domain file f in an AP_m area
$n_{m,f}$	the number of incoming requests for the content file f at AP_m

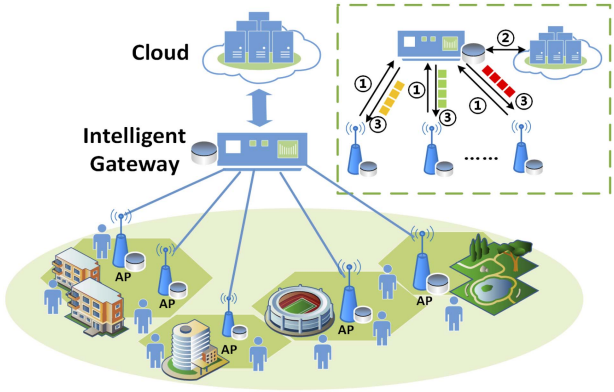


Fig. 1. The architecture of edge cache network.

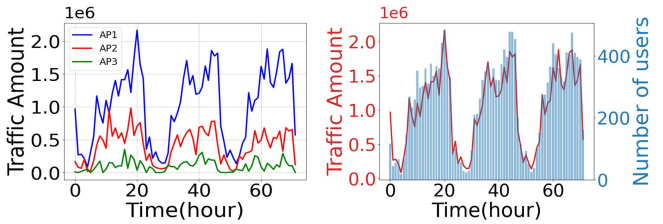
$\{f_1, f_2, f_3, \ldots, f_F\}$ denote the set of all the different files, which is located in the cloud. Additionally, the cache at the intelligent gateway and each AP can only store a subset of the total files owing to the limited cache capacity. It must be noted that the intelligent gateway connects to the cloud through large-capacity and high-speed links (e.g., optical fiber). Optical fibers-based wired communications offer a number of advantages, such as enormous bandwidth, low latency, low interference, low attenuation, and high reliability. Its wired propagation delay is significantly smaller compared with the delay from the end-user and the gateway [32], [33]. This is close to the current reality as most devices nowadays connect to Internet using Wi-Fi. According to a recent report from the Wi-Fi Alliance (Telecom Advisory Services, LLC [3] 2022), more than 18 billion Wi-Fi devices are currently deployed worldwide. We believe this number will be much bigger in the future.

Notably, fifth-generation wireless (5G) is the latest iteration of cellular technology designed to increase the speed and reduce the latency of wireless networks. 5G technology can achieve Ultra-Reliable Low-Latency Communication (URLLC) applications targeting sub-1 ms end-to-end latency [34]. The high speed, high capacity, and low latency of 5 G can manage the data transmission so that many intelligent terminal devices can exchange data in near real time. However, the above performance can be achieved only under perfect radio link conditions. As a wireless communication technology, 5 G cannot avoid interference issues, such as metal shielding and electromagnetic signal interference [35]. Besides, explosive growth of various intelligent terminal devices can cause the 5 G network to face massive data traffic shocks [36]. Moreover, in 5 G wireless networks, 5G-signal strength degrade [37] and user-perceived latency increases [38] when you move further from a broadcast station. Finally, because of the insufficient coverage and high cost of using 5 G services, most users nowadays still use Wi-Fi in the video streaming applications [16].

In this article, we primarily focus on caching at the Wi-Fi APs. The average latency of Wi-Fi 6 (the latest wireless architecture) is 20ms [39]. However, Wi-Fi signal is vulnerable to interference, causing unstable or slow wireless connections. Compared with Wi-Fi wireless communications, optical fiber-based wired communications can achieve stable data transmission, and the delay is μs-level [40]. Thus, the wireless backhaul link delay in Wi-Fi networks between the end-user and gateway aggravates the above delay gap [39]. Therefore, we ignore the wired transmission delay between the intelligent gateway and the cloud as in paper [41] and consider the transmission latency of a content file in terms of (1) the wireless transmission latency between the end-user and the Wi-Fi AP and (2) backhaul latency between the Wi-Fi AP and the gateway. We model this two-link latency in our paper. Then, we assume that content files are available at the intelligent gateway when they are requested. To simplify the analysis, we also assume that all the content files have the same size s_f, and each AP has the same cache capacity C_{AP} (bits) which is a multiple of s_f. This can be achieved by segment files into chunks [42]. The total number of different files in the network is $F{=}|\mathbf{F}| > M \cdot \frac{C_{AP}}{s_f}$.

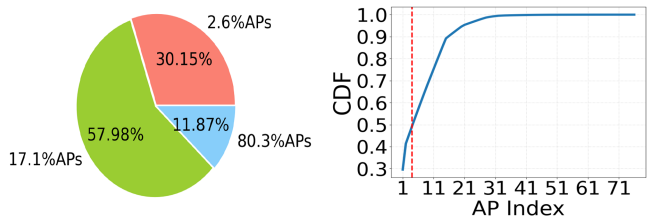
The intelligent gateway can also provide data collection, cache management, and content distribution. The cache management strategy conducts the following steps on an intelligent gateway: ① receiving end-user requests in APs and decide which content file needs prefetching and how to place the content file; ② prefetching the content file from the cloud; and ③ distributing the content file to the APs according to the content placement decision.

[3] http://www.teleadvs.com/



(a) Traffic pattern over the time (b) The number of user associations and the amount of traffic

Fig. 2. Pattern of traffic and user associations of three representative APs.



(a) Distribution of user associations at each AP (b) The CDF of the number of received requests for each AP

Fig. 3. Pattern of user associations and video requests of all APs in a week.

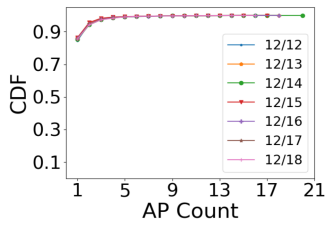


Fig. 4. The CDF of AP counts where a video content file is requested.

If the requested content is in the associated AP, the end-user can directly fetch it. Otherwise, the end-user obtains the content file through the intelligent gateway from the cloud, resulting in a high transmission delay. The detail of the delay is analyzed in Section II-C and evaluated in Section V-C. It must be noted that we investigate the edge cache under the current edge-cache network architecture without enabling the device-to-device (D2D) cooperation among APs (Please refer to Section II-B for details). When the popular content files are accurately predicted and placed in the AP, it can improve users' experiences by reducing the content delivery latency.

B. Analysis of a Youku Trace

In this study, we analyze a real-world trace from Youku, a major video service provider in China. Prior work [43], [44], [45], [46], [47] demonstrated how the data trace is originally produced and analyzed, including traffic data acquisition, metadata crawling/collection, and trace analysis. Moreover, the trace has been used in previous studies on edge caching strategies [28], [41], [48], [49]. The trace contains 41,125 video clips of 25 types and 155,991 video requests from 46,994 end-users sent to 76 APs. Each trace item records the anonymous user identifier, the AP identifier, timestamp, URL of the HTTP requests, video ID, and video type, which is collected from a provincial cellular network in northeast China between Dec 12, 2015 and Dec. 18, 2015.

First, we analyze temporal and spatial user associations and traffic usage patterns. Fig. 2(a)∼(b) show that different APs tend to have different numbers of end-user associations and traffic load patterns. We select three typical APs to show the traffic amount at each AP over the time, see Fig. 2(a). We observe an obvious difference among these APs. For example, the amount of traffic at AP_1 is much higher than that of AP_3. Fig. 2(b) shows that the amount of traffic and the number of user associations at one AP vary dramatically within a day, e.g., the amount of traffic and user associations are both concentrated around daytime, and decrease dramatically at late night and touches the bottom around 3 am. Moreover, the number of user associations is strongly correlated to the amount of traffic over the time. In addition, they are time-dependent.

Observation 1. Performing proactive content placement during off-peak periods. The above findings motivate us to place content files in advance during the low-traffic period, which avoids high traffic during the day and improves the QoE. In this article, we decide to update the cache once a day.

Second, we take a close look at the user association and video requests at each AP. Fig. 3 shows the distribution of user associations and the number of video requests of 76 APs in one week. More than 88% of end-users' requests are handled by only 19% of APs, see Fig. 3(a). Fig. 3(b) shows the cumulative distribution function (CDF) of the number of received requests for each AP. We find that 5.2% of the total APs received requests together count for 49.3% of the total requests, while more than 80% APs received only 10.8% of total requests. In summary, a few APs (e.g., the top 5.2% APs) serving a major portion of total requests implies that a large number of APs handle a very small number of user associations and requests.

Observation 2. Placing different numbers of content files at different APs. The above analysis indicates that caching the maximum number of content files in all APs may not be always the best choice considering the cost of cache placement, while caching less content in some APs may result in a better overall system performance. Thus, we need to place an appropriate amount of content files in the cache according to the possible requests at APs, aiming to avoid unnecessary waste of bandwidth and cache resources.

Third, we investigate the same file that has been requested at different numbers of APs. Fig. 4 shows the CDF of AP counts for all content files requested by end-users. The results unveil that most of the content files are requested by end-users from few APs. There are 86.7 percent of video files requested by end-users from one single AP, 97.3 percent of video files requested by end-users from no more than three APs. The rest of the video files are requested from more than three APs. This implies significantly different preference of various content files among APs. Next, we analyze the same video file requested from different Local Area Networks (LANs) per day. In Fig. 5, we observe that 92.9%

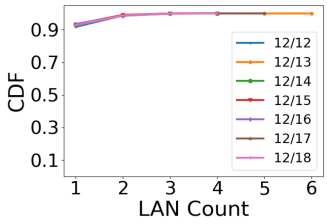
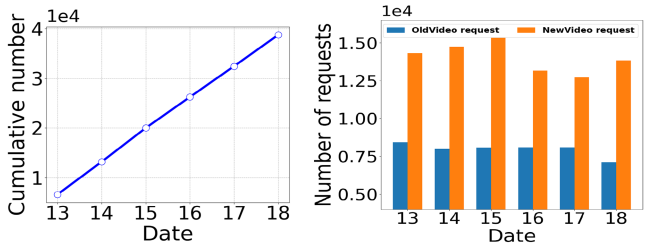


Fig. 5. The CDF of LAN counts where a video content file is requested.



(a) Cumulative number of new video content file in a certain week (between Dec. 13 and 18). (b) Number of new and old video requests in a certain week (between Dec. 13 and 18).

Fig. 6. The number of new video content files and its requests in a certain week (between Dec. 13 and 18).

of video files are requested by end-users within one single LAN, *i.e.,* different APs requesting the same video content may locate close to each other.

Observation 3. Caching content files with highest diversity at APs. The above discussion shows that different users associated to different APs may request different content files. In the trace used in our study, the overlaps between requested files in different APs are relatively low. Caching the same popular content files on multiple APs may gain little performance benefits but lead to wasting cache resources because of redundancy. Notably, in this article, we focus on optimizing performance for the majority of cases (over 86%) and caching one file at one AP. The random-access behaviors of users require to cache highly diverse files at different APs to better satisfy various requirements. Paper [26] also suggests the same concept as the highest diversity, *i.e.,* caching any content file only in one AP. This enables APs to cache the greatest number of different content files and simplifies the formulation of the cache placement problem.

Additionally, based on the above observation, different APs requesting the same video content may locate close to each other within the same LAN. We can take advantage of several recent developed technologies, such as dynamically formed cluster of APs [50], cache-aware user association [51], and novel new user-AP association mechanism [52] to relieve the concerns of caching one file only at one AP. These technologies allow users to associate to a specific AP, which not only provides good channel conditions, but also caches the user's request content. Emerging research can facilitate the caching of more content under multiple APs, thereby providing users with a higher probability of obtaining the requested contents locally from the caches of clustered APs.

It is helpful to predict the popularity of a specific video file in the proactive content placement during off-peak periods for alleviating backhaul congestion. However, it may not be easy to predict the popularity of a specific video file before it is cached in APs. The reason is that the popularity prediction is affected by various location-dependent factors (e.g., user preferences, the number of requests) and content-dependent factors (e.g., category, length, and frame quality for video contents). For example, the information collected at APs located in low-traffic areas is sparse, leading to an unsatisfactory accuracy of popularity prediction. Moreover, short-term bursts in the number of video requests may not accurately predict the future popularity of video files. Analyzing the number of days all video files were requested, we found that the majority (about 80%) of video files complete the growth in the number of user requests on the first day they are requested, with no user requests one day later. There were even cases such as a video file being requested 943 times within a minute after the first request and no more requested later. Therefore, we may need additional information to make an accurate popularity prediction.

In this work, we focus on predicting the popularity of new content files and take advantage of a lot of auxiliary information (e.g., the category attributes of files, implicit feedback of end-users, and different semantic information reflected by the heterogeneous information network).

In addition, collaboration among APs may bring other overheads, such as high communication costs and complex management mechanisms that include but are not limited to identifying appropriate APs to collaborate, maintaining a global or local content index table for data exchanges between neighboring APs, redirecting user-requests, etc. Therefore, collaborative caching is not considered in this study.

Fourth, we examine the number of new video content files and end-users requests over a week. Fig. 6(a) shows that the cumulative number of new video files continuously increases over the time, which implies that new video content files emerge daily. For an AP, we consider the content file that has been requested is the old content file.[4] As shown in Fig. 6(b), the percent of user requests for new video files is 63.7% on average per day. Further, the popularity of new content files[5] is difficult to predict because statistical information related to those files (e.g., the number of views) is not available.

Observation 4. Understanding end-users' preferences for new content files. The above phenomenon indicates that the preference of content files at APs varies over the time, and end-users incline to request new content files. We need to predict the new content popularity to improve the cache performance.

C. Problem Formulation

Let us assume that an end-user request a file in set $\mathbf{I} = \{f_1, f_2, f_3, \ldots, f_F\}$. If end-user u_k requests file i, we have

[4]The term *content file* is the same as *file*, unless otherwise stated. The term *'old content files'* and *'known domain content files'* have the same meaning.

[5]*The term 'new content files'* has the same meaning as *'unknown domain content files'*

$r_{k,i} = 1$; otherwise, we set $r_{k,i} = 0$. As mentioned in Observation 3, in this study, an AP can store multiple files while one file is only placed at an AP, aiming to improve cache diversity and avoid redundant files in APs. To study the cache content placement problem between content files and APs, we define a 0-1 cache placement matrix Φ consisting of entries $z_{m,f} \in \{0, 1\}$. The notation $z_{m,f} = 1$ denotes file f that is placed in AP_m.

As mentioned in Section II-A, we ignore the transmission delay between the intelligent gateway and the cloud in this work. The transmission latency of a content file consists of (1) the wireless transmission latency between the end-user and the Wi-Fi AP and (2) backhaul latency between the Wi-Fi AP and the gateway. Let $g_{k,m}$ denote the channel fading coefficient between AP_m and end-user u_k. Considering a time-varying channel, we employ a mean-reverting Ornstein-Uhlenbeck process [53], [54] to represent the evolution of $g_{k,m}$ as follows:

$$\mathrm{d}g_{k,m} = \frac{1}{2}\varsigma\left(\mu_g - g_{k,m}\right)\mathrm{d}t + \sigma_g \mathrm{d}B_{k,m} \tag{1}$$

where ς denotes the speed of the process toward the long-term mean. The positive numbers μ_g and σ_g denote the long-term mean and variance of the process, respectively. $B_{k,m}$ is the standard Brownian motion. Let us assume that the channel path loss between end-users and APs is fixed. By combining channel fading and path loss, we get $|h_{k,m}|^2 = |g_{k,m}|^2 dis_{k,m}^{-\alpha}$, where α denotes the path loss exponent and $\alpha \geq 2$. The symbols $|h_{k,m}|^2$ and $dis_{k,m}$ denote the channel gain and euclidean distance between AP_m and end-user u_k, respectively.

When requested files are cached in nearby APs, end-users receive file *i* from the corresponding APs through wireless links. Then, the transmission rate from AP_m to end-user u_k being served can be expressed as

$$\upsilon_{k,m} = w \log_2\left(1 + \frac{|h_{k,m}|^2 P_m}{\sigma^2 + \sum_{j\neq m, j\in\mathbf{M}} |h_{k,j}|^2 P_m}\right) \tag{2}$$

where w, P_m and σ^2 denote the transmission bandwidth, transmission power of AP_m, and background noise power, respectively. Please note that we mainly focus on edge caching content placement in this study; therefore, other factors (e.g., power allocation, wireless channel allocation, traffic control, and protocol overheads) in the network are not our focus. We also believe that our proposed algorithm can adapt to different delay models to find optimal solutions. In the future, we will consider more factors in the network layer to study its relation to the edge cache performance. Thus, the wireless transmission delay between end-user u_k and AP_m is denoted as

$$\tau_{\mathrm{k,i}}^1 = \frac{s_f}{\upsilon_{k,m}} \tag{3}$$

If a cache miss occurs when end-user u_k requests files at APs, the end-user requests are transmitted through the intelligent gateway that connects all APs via a backhaul link. We denote the backhaul delay of end-user u_k connected to AP_m as $\mathrm{D}_{k,m}^B$. For a backhaul link, the latency depends on the average link distance, average traffic load, and average number of APs connected to the gateway. It can be modelled as an exponentially distributed random variable with a mean value of D^B [55]. The backhaul latency of end-user u_k to obtain file i through AP_m is expressed as

$$\tau_{\mathrm{k,i}}^2 = \mathrm{D}_{k,m}^B\left(1 - z_{m,f}\right) \tag{4}$$

Thus, we can obtain the transmission latency of file i as

$$d_{k,i}^m = \tau_{\mathrm{k,i}}^1 + \tau_{\mathrm{k,i}}^2 = \frac{s_f}{\upsilon_{k,m}} + \mathrm{D}_{k,m}^B\left(1 - z_{m,f}\right) \tag{5}$$

The association between an end-user and AP can be expressed by a binary variable $l_{m,k} \in \{0, 1\}(\forall m \in \mathbf{M}, \forall k \in \mathbf{U})$. We optimize the cache content placement function Φ using an intelligent gateway to place the content file in the cache of APs, thereby minimizing the average content delivery delay. The problem is described as follows.

$$\begin{aligned}
&\min_{\Phi} \frac{1}{K}\sum_{k\in\mathbf{U}}^{K}\sum_{m\in\mathbf{M}}^{M}\sum_{i\in\mathbf{I}}^{F} r_{k,i} l_{m,k} d_{k,i}^m \\
&\text{s.t. } C1: z_{m,f} \in \{0,1\}, \forall m \in \mathbf{M}, \forall f \in \mathbf{F} \\
&\quad C2: l_{m,k} \in \{0,1\}, \forall m \in \mathbf{M}, \forall k \in \mathbf{U} \\
&\quad C3: r_{k,i} \in \{0,1\}, \forall k \in \mathbf{U}, \forall i \in \mathbf{I} \\
&\quad C4: \sum_{f\in\mathbf{F}} z_{m,f} \leq \frac{C_{AP}}{s_f} < F, \forall m \in \mathbf{M} \\
&\quad C5: \sum_{m\in\mathbf{M}} z_{m,f} \leq 1, \forall f \in \mathbf{F}
\end{aligned} \tag{6}$$

where $d_{k,i}^m$ denotes the transmission latency for end-user u_k to obtain content file *i* from its coverage AP_m. Its access depends on the content cache state, which is determined by the cache placement variable $z_{m,f}$. Constraints C1∼C3 guarantee $z_{m,f}$, $l_{m,k}$, and $r_{k,i}$ to be binary variables. Constraint C4 ensures the maximum number of files that an AP can store, and constraint C5 ensures that each file will be placed at only one AP to avoid redundant caching.

The optimization problem (6) is a Helper decision problem (*i.e.,* which file should be cached by which helper) where an AP is a helper. In [56], this problem has been proven to be an NP-hard combinatorial problem, and its complexity increases exponentially with the number of content files and APs. In Section IV-B, we propose a many-to-one file placement matching algorithm for optimizing the placement matrix Φ to solve the cache content placement problem.

III. Design of GACD

We describe the design of GACD in this section. We first describe the system overview and its workflow in Section III-A. Then, we demonstrate the two components of GACD in Sections III-B and III-C in detail.

A. System Overview

Fig. 7 shows a system overview. GACD takes file attributes and end-user requests at APs collected by the intelligent gateway as the input. Subsequently, it outputs the cache deployment

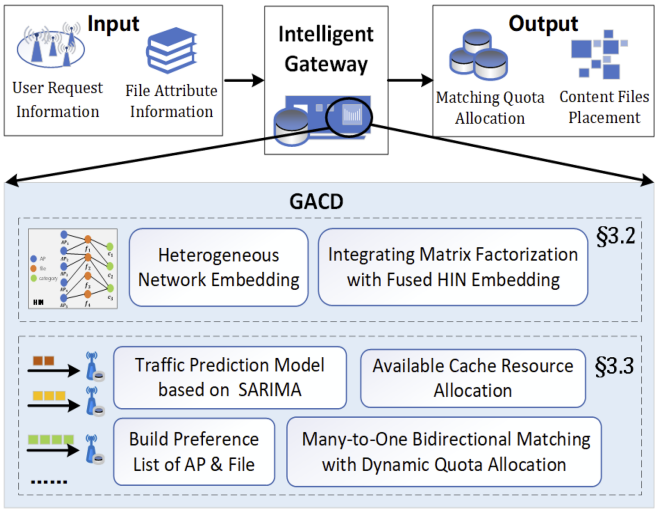


Fig. 7. System overview of GACD.

policy, including dynamic matching quota allocation and content placement decisions.

Different APs contain various user-AP associations and traffic consumption in the network. First, we study the changes in content popularity at the AP-level, and predict the popularity of new content files, aiming to improve the cache-hit rate. Second, we model the content placement problem as a matching game with dynamic quota allocation. Subsequently, we obtain stable AP-file pairs for cache-content placement. Specifically, we apply the seasonal autoregressive integrated moving average (SARIMA) model to predict the future traffic usage of each AP and obtain an appropriate matching quota based on the *on-demand allocation rule*. The content file can be effectively placed in the cache based on the allocated quota. Next, we present the details of GACD in Fig. 7.

B. Predict the AP-Level Popularity of Content Files

Before executing the content-placement strategy, the intelligent gateway predicts the AP-level popularity based on the end-users' requests and file attributes. Let $p_{m,f}$ be the popularity distribution at the AP-level that indicates the popularity of the content file f in an AP_m area. First, we process the popularity distribution of known domain content files that have already been requested. Second, we predict incoming end-user requests for unknown domain content files, including newly released content files or content files that have not appeared in local history records. This process is called unknown domain files popularity prediction, which predicts whether end-users have potential preferences for content files that have not been requested.

We evaluate the AP-level content popularity of known domain files based on the implicit feedback of end-users (e.g., number of clicks or views) as follows.

$$\bar{p}_{m,f} = \frac{n_{m,f}}{\sum_{f \in \mathbf{F}} n_{m,f}} \tag{7}$$

where $n_{m,f}$ is the number of incoming requests for the content file f at AP_m, $\sum_{f \in \mathbf{F}} n_{m,f}$ is the total number of arriving requests for all content files at AP_m, and $f \in \mathbf{F}$ is the known domain file. The methods used to process unknown-domain content files may not be the same as those used to process known-domain content files. This is because statistical information related to unknown-domain content files (e.g., number of views) is unavailable. However, unknown-domain content files contain basic attribute information such as video ID, time of release, and category. This heterogeneous and complex auxiliary information can be integrated using HIN-based information modeling methods to increase prediction accuracy. Furthermore, as indicated by observations in Section II-B, the percent of user requests for new video files is 63.7% on average per day; therefore, the approach to predicting the popularity of these unknown-domain content files and caching the most popular content can improve the cache hit ratio and cache performance. In this module, we adopt a heterogeneous information network [57] to predict potential user preferences. As a powerful information modeling method, heterogeneous information networks have been widely applied to integrate heterogeneous and complex information, owing to their flexibility in modeling data heterogeneity. Its underlying data structure is a directed graph that contains multiple types of objects or multiple types of links. First, we build the heterogeneous information network *HIN* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ to extract information. Symbols $\mathbf{AP} \in \mathcal{V}$ and $\mathbf{F} \in \mathcal{V}$ are the set of APs and content files, respectively. The triple $< m, f, \bar{p}_{m,f} >$ represents a record of AP-level content popularity $\bar{p}_{m,f}$ of known domain files f at AP_m. $\mathbf{R} = \{< m, f, \bar{p}_{m,f} >\} \in \mathcal{E}$ is a set of popularity records. Please refer to Section IV-A1 for details. Given $\mathbf{AP} \in \mathcal{V}, \mathbf{F} \in \mathcal{V}$, $\mathbf{R} \in \mathcal{E}$, and the heterogeneous information network $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, we can apply a network embedding method [58] to obtain the vector embeddings of APs and files and then apply a personalized non-linear fusion function to integrate these embeddings into a unified form. Subsequently, we extend the classic matrix factorization model by incorporating the learned embeddings. Finally, we obtain the popularity distribution of the unknown domain files $\widehat{p_{m,f}}$, where we have $m \in \mathbf{AP}$ and the unknown domain file $f \in \mathbf{F}$.

C. Content Placement as a Matching Game With Dynamic Quota Allocation

We apply the matching game theory to address the problem of content placement. The matching theory can solve the NP-hard combinatorial problem [59]. A matching procedure is applied between the content file in $\mathbf{F}$ and the AP in $\mathbf{M}$. An AP stores multiple files, S_m is the matching quota of AP_m; the set of matching quota in all APs is represented as $\mathbf{S} = \{S_1, S_2, S_3, \ldots, S_M\}$, where $S_m \leq \frac{C_{AP}}{s_f} < F$. As indicated by observations in Section II-B, caching few files in some APs that are associated with few end-users improves limited cache resource utilization. Thus, our goal is to allocate the appropriate quota of each AP that does not exceed its demand, which can avoid the waste of bandwidth and cache resources. By solving this content-placement problem, we can determine (1) the number of content files that should be placed on each AP, and (2) the specific content file that should be cached at a specific AP.

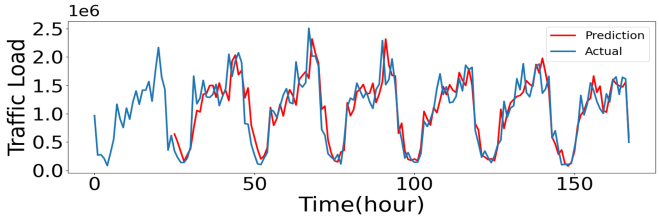


Fig. 8. Traffic load of a certain AP over the time measured using trace from Youku dataset.

1) Traffic-Prediction Model: There are various user-AP associations and traffic consumption in different APs. Fig. 2(b) shows the traffic utilization and the number of connected end-users at an AP. The daily traffic usage at the AP, which is a time-series process, varies over the time. We analyze the past observations of the same variable and build a time series prediction model. Subsequently, the model can be used to predict future trends. As shown in Fig. 8, the actual traffic time series exhibit seasonal patterns. SARIMA model is a popular and accurate approach to forecast traffic data with a strong seasonality trend [60]. In addition, the SARIMA model can satisfactorily describe non-stationary time series by neglecting the random fluctuations. Therefore, we adapt the SARIMA model to predict the traffic amount based on the obtained pattern of the time series. We predict how much network traffic will be utilized at an AP based on the SARIMA model, and the model structure is $(p,d,q)(P,D,Q)_s$. Then, we have

$$\phi_p(B)\Phi_P\left(B^s\right)(1-B)^d\left(1-B^s\right)^D X_t = \theta_q(B)\Theta_Q\left(B^s\right)\varepsilon_t \tag{8}$$

where Φ and Θ are the seasonal autoregressive and moving average parts, and ϕ and θ are the non-seasonal autoregressive and moving average parts. B is the backward-shift operator, ε_t is the estimated residual at time t, and s is the number of periods in a season. Non-negative integers *p, d, q, P, Q,* and *D* are the polynomial orders of the auto-regressive (AR), integral (I), and moving average (MA) of the non-seasonal and seasonal components in this model, respectively. We predict traffic load by learning the historical data in the previous 24 h. In this study, we have $(p,d,q)(P,D,Q)_s = (0,1,1)(0,1,1)_{24}$. There are 24 periods, one for each hour of the day. We employ users' requests and traffic at one AP to predict the future traffic load over the time. Fig. 8 presents prediction results. The mean absolute percent error of the prediction is 18.41%, which suggests that the SARIMA model can achieve acceptable accuracy for traffic-load estimation at each AP.

2) The On-Demand Allocation Rule: We use SARIMA model to predict traffic load of each AP and understand its trend. Subsequently, we effectively manage cache resources and avoid resource wastage by adjusting the matching quota of each AP. Specifically, we must determine an appropriate *allocation rule*, *i.e.,* how to assign quota for each AP based on the corresponding demands. Herein, the 'demand' is the number of user-request content files that each AP may receive in the future. An *allocation rule* is a function, whose input parameters include the demands of each AP and the available cache capacity, and its output parameter is the matching quota allocated to each AP.

Let $\mathbf{T}$ be the future traffic consumption of each AP and define $\mathbf{T} = \{T_1, T_2, T_3, \ldots, T_M\}$ as the traffic-prediction result. We define *on-demand allocation rule* as follows.

$$S_m = \min_{m \in \mathbf{M}} \left(C_{AP}/s_f, \lceil T_m/s_f \rceil\right) \tag{9}$$

where C_{AP}/s_f denotes the maximum number of content files that each AP can store. $\lceil T_m/s_f \rceil$ denotes the maximum demands of each AP for content files. When the demand of an AP is higher than its available cache capacity, we fully use everything to satisfy as many user requests as possible. To avoid wasting resources in the cache, we allocate a small-size quota in the AP whose demand is less than the available cache capacity.

3) Content Placement: We introduce a many-to-one bidirectional matching game with dynamic quota allocation in this module. File set $\mathbf{F}$ and AP set $\mathbf{M}$ are finite disjoint sets with $i \geq 1$ and $j \geq 1$ elements. Each file $i \in \mathbf{F}$ matches one AP at most, whereas each AP $j \in \mathbf{M}$ can hold one or more files based on its matching quota S_m. Matching theory [59] enables each participant to define its preference list in the matching process. We define the file-AP matching as follows.

Definition 1: Given two finite and disjoint sets of files $\mathbf{F}$ and APs $\mathbf{M}$. There are two disjoint finite sets of $\psi^{\mathbf{F}} = \{1, \ldots, |\mathbf{F}|\}$ and $\psi^{\mathbf{M}} = \{1, \ldots, |\mathbf{M}|\}$. Therefore, a many-to-one matching function $\Psi : \{\mathbf{F}\} \cup \{\mathbf{M}\} \to \{\mathbf{F}\} \cup \{\mathbf{M}\}$ is defined for all $i \in \psi^{\mathbf{F}}$ and $j \in \psi^{\mathbf{M}}$.

$$\begin{aligned}
&(1)\Psi\left(F_i\right) \in \left(M_{j' \in \psi^{\mathbf{F}}} \in \mathbf{M}\right) \text{ and } \left|\Psi\left(F_i\right)\right| \in \{0,1\}.\\
&(2)\Psi\left(\mathrm{M}_j\right) \subseteq \left\{F_{i' \in \psi^{\mathbf{M}}} \in \mathbf{F}\right\} \text{ and } \left|\Psi\left(\mathrm{M}_j\right)\right| \leq S_j.\\
&(3)\Psi\left(F_i\right) = \mathrm{M}_j \Leftrightarrow \Psi\left(\mathrm{M}_j\right) = F_i.
\end{aligned} \tag{10}$$

We define the many-to-one matching function with dynamic quota allocation Ψ as a three-tuple $\Psi : (\mathbf{F}, \mathbf{M}, S_m)$, in which S_m is obtained from (9). Condition (1) implies that each member of $\mathbf{F}$ can be matched to at most one member of $\mathbf{M}$, condition (2) implies that each member of $\mathbf{M}$ can be matched to multiple member of $\mathbf{F}$, and condition (3) implies that if F_i is matched to M_j, then M_j is also matched to F_i. The binary relationship between the file in $\mathbf{F}$ and AP in $\mathbf{M}$ is defined as the preference relationship ($\succ$). When file F_i is placed in AP M_j, we have $\mathrm{M}_j \succ_{F_i} \mathrm{M}_{j'}$. Additionally, the preferences of AP M_j for end-users can be expressed as $\mathrm{F}_i \succ_{M_j} \mathrm{F}_{i'}$. Noted that the matching relationship is built only if we have $\Psi(F_i) = \mathrm{M}_j$ and $\Psi(\mathrm{M}_j) = F_i$, which guarantees that the matching game is a bilateral agreement.

D. Extended Application and Open Research Issues

There are a set of open research issues and extended applications that should be investigated in future.

Open Research Issues. First, because of the difficulty in finding real-world traces, the design of GACD is based on only one specific trace. We need to test the performance of GACD in different application scenarios and investigate how to intelligently adapt the caching strategy according to the new user requests patterns. Second, in the current design of GACD, we did not consider the diverse resolutions at user devices. It

is worth exploring caching algorithms that tradeoff the cache storage cost and communication cost by adapting to different device configurations. Third, currently, each file is cached only at one edge node to reduce the redundancy. However, a certain level of redundancy may help further improve the hit rate. We will study the tradeoff between cache diversity and redundancy to enhance cache performance. Forth, we can identify the users' file preference by digging deep into their file request history. However, this may be intrusive to user's privacy. We will study the integration of privacy-preserved user preference analysis in caching algorithm design. For example, privacy protection methods (e.g., blockchain-based privacy-preserving model [61] and federated learning-based methods [62]) could be exploited for secure caching at the edge. Fifth, when a file is cached in more than one AP, we need to research what will be the appropriate number of APs and which set of APs. All these are interesting research problems to explore. In this article, we take out the initial step to investigate the edge caching problem, and we will definitely extend our work to investigate the above research questions. In addition, in the design of GACD, we assume the users are static. When it is applied in mobile systems, new caching placement algorithms should consider the impact of user mobility.

Extended Application of GACD. Besides their application in video streaming, the proposed mechanism can be applied in many IoT-related applications. For example, GACD can be deployed at self-driving vehicles and roadside units in a vehicular network for latency-sensitive vehicular applications such as on-vehicle entertainment. GACD can identify and cache popular content based on end-users' interests, thereby providing an excellent in-vehicle entertainment experience. GACD can also be applied to determine the content and location of caching files in augmented reality and virtual reality (AR/VR) applications. For example, in AR-enabled museum touring, GACD can cache high-definition images, maps, and high-resolution/framerate videos that are frequently accessed, for low-latency and high-quality service.

IV. Algorithms in GACD

We present the unknown domain files popularity prediction algorithm, which uses all end-users' requests for files and the category attributes of files to predict end-users' preferences. Thereafter, we introduce a many-to-one file placement matching algorithm to achieve the stable matching of APs and files.

A. unknown Domain Files Popularity Prediction Algorithm

Fig. 9 shows the process of the popularity of unknown domain files prediction. In this study, the unknown-domain file popularity prediction problem is considered as a similarity search task over the HIN. The popularity prediction process is based on low-dimensional embeddings learned from the *HIN*, which characterize the correlations between known- and unknown-domain files. First, we model the known information to build a heterogeneous information network *HIN* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Second, using a network embedding method, we extract information from *HIN* to obtain the final embeddings of APs and files. Third,

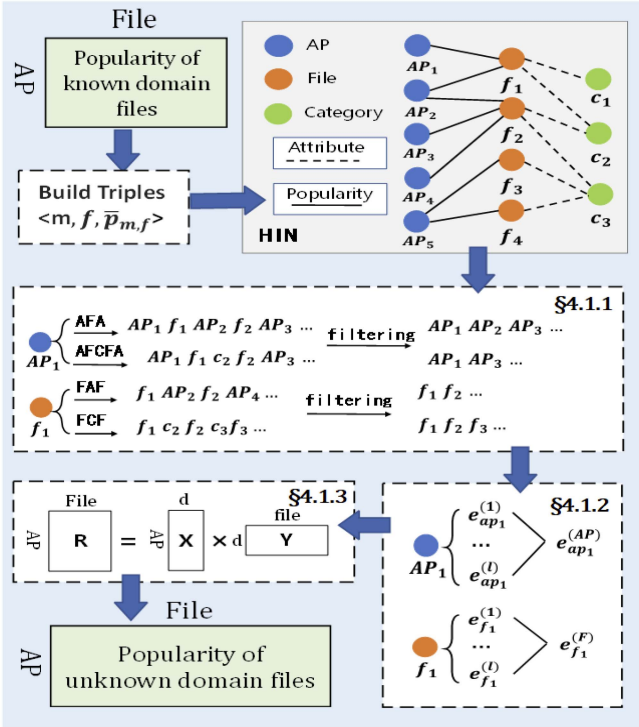


Fig. 9. Unknown domain files popularity prediction.

we conduct a prediction procedure using the embedding learned from *HIN*.

1) meta-Path Based Random Walk in HIN: Fig. 9 shows the heterogeneous information network used in this study. *HIN* contains multiple types of objects, such as APs, files (f), categories (c) and different types of links (*i.e.,* the popularity of known domain files at APs and the attribute of file categories). From *HIN*, we can find different semantics information, such as feedback (*i.e.,* AP-file) and attributes (*i.e.,* file-category). Two objects in the *HIN* can be connected via different paths, called meta-path [57]. Meta-path is an important concept to characterize the semantic patterns for HINs, which is similar to a feature in a traditional data set [63]. For example, in Fig. 9, two APs are connected via *'AP-File-AP'* (AFA) path and *'AP-File-Category-File-AP'* (AFCFA) path. Two files are connected via *'File-AP-File'* (FAF) path and *'File-Category-File'* (FCF) path. Different semantics information is enclosed in these paths. The AFA path means that end-users associated with different APs request the same content file. The AFCFA path represents that end-users associated with different APs request content files within the same category. FAF denotes that end-users request different content files under the same AP coverage area. FCF expresses different content files belonging to the same category.

We use a meta-path based random walk method [64] to generate heterogeneous neighborhoods for each node, aiming to capture the complex semantics reflected in *HIN*. We take the *HIN* in Fig. 9 as an example. Given a meta-path AFA, we can generate three sample walks (*i.e.,* node sequences) by starting with AP_2: 1) $AP_2 \longrightarrow f_1 \longrightarrow AP_1$, 2) $AP_2 \longrightarrow f_2 \longrightarrow AP_3$, and 3) $AP_2 \longrightarrow f_2 \longrightarrow AP_4$. Similarity,

given a meta-path AFCFA, we can also generate another node sequence: $AP_2 \longrightarrow f_2 \longrightarrow c_3 \longrightarrow f_3 \longrightarrow AP_5$. Different meta-paths can produce meaningful node sequences corresponding to different semantic relationships.

We aim to predict the AP-level content popularity of unknown domain files; therefore, the focus of this study is to learn effective embeddings of APs and files. Hence, we merely select meta-paths starting with *AP type* or *file type* rather than other types of objects. However, the node sequences using the random walk method contain other nodes with different types; the final embedding are learned using homogeneous neighborhood nodes. Thus, we further filter nodes whose type is different from the starting node type. The final sequence should only contain nodes of the same type as the starting node. Taking the meta-path of AFA as an example, we can obtain a sampled sequence '$AP_1 \longrightarrow f_1 \longrightarrow AP_2 \longrightarrow f_2 \longrightarrow AP_3$' starting with *AP type*. Then, we apply type filtering to obtain homogeneous node sequence '$AP_1 \longrightarrow AP_2 \longrightarrow AP_3$'. Subsequently, we use homogeneous neighborhood nodes to learn the final embeddings.

2) Heterogeneous Network Embedding and Personalized Non-Linear Fusion: The network embedding aims at learning a low-dimensional embedding $e_v \in \mathbb{R}^d$ for each node $v \in \mathcal{V}$. We use HIN embedding [58] to extract useful information from *HIN*. Given node $v \in \mathcal{V}$, we can obtain a set of embeddings $\{e_v^{(l)}\}_{l=1}^{|\mathcal{P}|}$, where $\mathcal{P}$ denotes the set of meta-paths, and $e_v^{(l)}$ denotes the representation of v at the lth meta-path. In this study, we obtain a set of embeddings $\{\boldsymbol{e}_{ap}^{(l)}\}_{l=1}^{|\mathcal{P}^{AP}|}$ and $\{\boldsymbol{e}_f^{(k)}\}_{k=1}^{|\mathcal{P}^F|}$ for each AP and file, where $\boldsymbol{e}_{ap}^{(l)} \in \mathbb{R}^d$ is the embedding of ap with regard to the l^{th} meta-path, and $\boldsymbol{e}_f^{(k)} \in \mathbb{R}^d$ is the embedding of f with regard to the k^{th} meta-path.

Afterward, we apply a personalized non-linear fusion function $g(\cdot)$ to transform the learned network embeddings of APs and files into a unified form. The integrating process is represented as

$$e_{\text{ap}}^{(\text{AP})} \leftarrow g\left(\left\{\boldsymbol{e}_{ap}^{(l)}\right\}_{l=1}^{|\mathcal{P}^{AP}|}\right) = \sigma\left(\sum_{l=1}^{|\mathcal{P}^{AP}|} w_{ap}^{(l)} \sigma\left(\mathbf{M}^{(l)} e_{ap}^{(l)} + \boldsymbol{b}^{(l)}\right)\right) \tag{11}$$

$$e_{\text{f}}^{(\text{F})} \leftarrow g\left(\left\{\boldsymbol{e}_{f}^{(k)}\right\}_{k=1}^{|\mathcal{P}^{F}|}\right) = \sigma\left(\sum_{k=1}^{|\mathcal{P}^{F}|} w_{f}^{(k)} \sigma\left(\mathbf{M}^{(k)} e_{f}^{(k)} + \boldsymbol{b}^{(k)}\right)\right) \tag{12}$$

where $e_{\text{ap}}^{(\text{AP})}$ and $e_{\text{f}}^{(\text{F})}$ are the final embeddings of AP ap and content file f, respectively. $\sigma(\cdot)$ is a non-linear function. $\mathcal{P}^{AP}$ and $\mathcal{P}^{F}$ are sets of meta-paths for APs and files, respectively. $\mathbf{M}^{(l)} \in \mathbb{R}^{D\times d}$ and $\mathbf{M}^{(k)} \in \mathbb{R}^{D\times d}$ are the transformation matrices for the AP and file, respectively. $\boldsymbol{b}^{(l)}$ and $\boldsymbol{b}^{k}$ are bias vectors. Each AP is assigned with a weight vector on meta-path $w_{ap}^{(l)}$, which indicates the preference weight of AP ap on its l^{th} meta-path. Similarly, $w_f^{(l)}$ is the preference weight of file f on the l^{th} meta-path.

3) Integrating Matrix Factorization With Fused HIN *Embedding for the Unknown Domain Files Popularity Prediction:* We use the embeddings learned from *HIN* in the prediction procedure. First, we build a predictor based on the matrix factorization (MF) model [65], which decomposes the statistical probability matrix into two low-rank matrices. The popularity of the unknown domain file f at AP_m is computed as

$$\widehat{p_{m,f}} = \mathbf{x}_m^\top \cdot \mathbf{y}_f \tag{13}$$

where $\mathbf{x}_m \in \mathbb{R}^D$ and $\mathbf{y}_f \in \mathbb{R}^D$ represent the latent factors of AP_m and file f, respectively. Since we have already obtained the embeddings for AP_m and file f, we further incorporate them into the the predictor as follows.

$$\widehat{p_{m,f}} = \mathbf{x}_m^\top \cdot \mathbf{y}_f + \alpha \cdot e_{\text{ap}}^{(\text{AP})^\top} \cdot \gamma_f^{(F)} + \beta \cdot e_{\text{f}}^{(\text{F})} \cdot \gamma_{ap}^{(AP)^\top} \tag{14}$$

where $e_{\text{ap}}^{(\text{AP})}$ and $e_{\text{f}}^{(\text{F})}$ are the fused embeddings. $\gamma_{ap}^{(AP)}$ and $\gamma_f^{(F)}$ represent the AP-specific and file-specific latent factors, respectively, which are paired with the HIN embedding $e_{\text{ap}}^{(\text{AP})}$ and $e_{\text{f}}^{(\text{F})}$. α and β are tuning parameters that integrate the three terms. The loss function can be expressed as follows.

$$\begin{aligned}\mathcal{L} = \sum_{\langle ap,f,\bar{p}_{m,f}\rangle \in \mathcal{R}} \left(\bar{p}_{m,f} - \widehat{p_{m,f}}\right)^2 + \lambda \sum_{AP} \Bigg(\|\mathbf{x}_{ap}\|_2 + \|\mathbf{y}_f\|_2 \\ + \left\|\gamma_{ap}^{(AP)}\right\|_2 + \left\|\gamma_f^{(F)}\right\|_2 + \left\|\Theta^{(AP)}\right\|_2 + \left\|\Theta^{(F)}\right\|_2 \Bigg)\end{aligned} \tag{15}$$

where $\widehat{p_{m,f}}$ is the predicted result in (14). $\Theta^{(AP)}$ and $\Theta^{(F)}$ are the integration function $g(\cdot)$ parameters of the APs and files, respectively. λ is the regularization parameter. The optimization of the loss function in (15) is implemented using the stochastic gradient descent strategy. Following [58], we set the embedding dimension number d to 64. The tuning parameters α and β are set to 1.0 and 1.0. We set the number of random walk $\varphi = 10$. The length for random walk t is 40. The size of neighbor κ is set to 3. The preference weight is set to 0.1. We set the number of latent factors D for all MF-based methods to 10.

We present this process in Algorithm 1, which contains three primary parts: (1) process known domain files, the computational complexity of obtaining $\bar{p}_{m,f}$ is $O(\pi)$. (2) the network embedding, aiming to learn the embeddings for APs and files. It processes a $O(|\mathcal{P}| \cdot \varphi \cdot t \cdot \kappa \cdot d \cdot (|\mathbf{AP}| \cdot log|\mathbf{AP}| + |\mathbf{F}| \cdot log|\mathbf{F}|))$ time complexity, where $|\mathcal{P}|$ is the number of selected meta-paths, φ is the number of random walk, t is the length of random walk, κ is the size of neighbor, d is the embedding dimension, and the number of APs and files is $|\mathbf{AP}|$ and $|\mathbf{F}|$, respectively. (3) the matrix factorization. In this part, the updating of $\mathbf{x}_m, \mathbf{y}_f, \gamma_{ap}^{(AP)}, \gamma_f^{(F)}$ takes $O(D)$ time complexity, where D is the number of latent factors. And the time complexity of updating $\Theta_{ap}^{(AP)}, \Theta_f^{(F)}$ is $O(|\mathcal{P}| \cdot D \cdot d)$. Thus, the time complexity is $O(\pi + |\mathcal{P}| \cdot \varphi \cdot t \cdot \kappa \cdot d \cdot (|\mathbf{AP}| \cdot log|\mathbf{AP}| + |\mathbf{F}| \cdot log|\mathbf{F}|) + D + |\mathcal{P}| \cdot D \cdot d)$.

B. A Many-to-One File Placement Matching Algorithm

We propose a many-to-one file placement matching algorithm to address the optimal content-placement problem (see (6)). The many-to-one matching mechanism consists of two game players

Algorithm 1: A unkNown Domain Files Popularity Prediction Algorithm.

Require: Implicit feedback, attribute information of files
Ensure: the popularity distribution of content files at AP-level, $\mathbf{ProcessknowndomainContentFiles}$
1: Obtain the incoming popularity distribution $\bar{p}_{m,f}$ of *known domain* files at AP-level according to (7);
$\mathbf{ProcessUnknowndomainContentFiles}$
2: Build *HIN* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, $\mathbf{AP} \in \mathcal{V}, \mathbf{F} \in \mathcal{V}, \mathbf{R} = \{< m, f, \bar{p}_{m,f} >\} \in \mathcal{E}$;
3: Use Heterogeneous network embedding to obtain the final embedding of APs($e_{\text{ap}}^{(\text{AP})}$) and files($e_{\text{f}}^{(\text{F})}$);
4: Integrating matrix factorization with fused *HIN* embedding;
5: Obtain the popularity distribution $\widehat{p_{m,f}}$ of *Unknown domain* files at AP-level according to (14);
6: Summarize the above results to obtain the AP-level popularity of all files $p_{m,f}$

(file $\mathbf{F}$ and AP $\mathbf{M}$). In the proposed matching game, one player must rank the other based on its preference list. Subsequently, the player achieves the potential matches based on its ranking information.

For the content files, we should place more popular files on APs with a large number of user requests. Herein, we obtain the total number of user requests at each AP using the connection matrix $\mathbf{L} = (l_{i,j})_{M \times K}$ and the number of requests sent by end-user u_k in the historical records n_k. We analyze the traffic weight[6] of APs $\mathbf{W} = [w_1, \ldots, w_i, \ldots, w_M]$ as

$$w_i = \frac{\sum_{k \in \mathbf{U}} n_k \cdot l_{i,k}}{\sum_{k \in \mathbf{U}} \sum_{i \in \mathbf{M}} n_k \cdot l_{i,k}} \tag{16}$$

where we have $w_i \in [0, 1]$ and $1 \leq i \leq M$. For file *f*, we rank all APs and build a preference list $\mathcal{PL}_f(M)$ as

$$\mathcal{PL}_f(M) = \text{sorted}(\mathbf{W}) \tag{17}$$

Similarly, each AP favors to cache the most popular content files locally. Since we obtain the AP-level content popularity $p_{m,f}$, we rank the popular files for each AP to build the preference list $\mathcal{PL}_m(F)$ as follows.

$$\mathcal{PL}_m(F) = \text{sorted}(\mathbf{p_{m,f}}) \tag{18}$$

where AP_m builds its preference list $\mathcal{PL}_m(F)$ based on the AP-level popularity sequence of content files. Function $sorted(\cdot)$ is used to sort results in (17) and (18). Noted that if the traffic weight of two APs or the popularity of two files is equal, their preference ranking is randomly determined.

When the preference list of the matching parties is built, we must design a stable matching solution to solve the content-placement problem (6). This procedure is illustrated in Algorithm 2.

[6]The traffic weight determines whether an AP is hot.

Algorithm 2: A Many-to-One File Placement Matching Algorithm.

Require: the capacity of each AP: C_{AP}, the size of each content file: s_f, the AP-level popularity of all files: $p_{m,f}$, the traffic weight of APs: $\mathbf{W}$,
Ensure: Stable matching AP-file pairs
1: **Initialization:** Unmatched file set $F_{un} = \mathbf{F}$
2: **for** $AP_m \in \mathbf{M}$ **do**
3: Obtain the future traffic consumption of each AP
4: using SARIMA model
5: Obtain the matching quota of APs based on (9)
6: Obtain preference list of files based on (17)
7: Obtain preference list of APs based on (18)
8: **while** $F_{un} \neq \emptyset$ **do**
9: Files proposed to APs:
10: **for** $f_i \in F_{un}$ **do**
11: Propose to the first AP m_j in its preferences $\mathcal{PL}_f(M)$ and set $z_{m,f} = 1$
12: Remove m_j from $\mathcal{PL}_f(M)$
13: Decision made by the APs:
14: **for** $m_j \in \mathbf{M}$ **do**
15: **if** $\sum_{f \in \mathbf{F}} z_{m,f} \leq S_j$ **then**
16: m_j keeps all the proposed content files
17: Remove f_i from F_{un}
18: **else**
19: m_j keeps the most preferred files
20: and rejects the remaining
21: Remove the most preferred files from the F_{un}
22: Add the rejected files into F_{un}
23: and set corresponding $z_{m,f} = 0$
24: Stop the algorithm when all files are placed on the AP or the quota of all APs is occupied
25: Obtain the optimal matching pairs

We propose a Gale-Shapely based algorithm to address the many-to-one file placement matching problem in GACD. In traditional matching problems [66], these preferences are static. However, in our matching game, the preference lists of APs and content files change dynamically according to the traffic weight and popularity distribution (see (17) and (18)). Additionally, the matching quotas of players are generally fixed in prior research on matching games [67], [68], [69]. In this study, we aim to adjust the matching quota of each AP dynamically, thereby avoiding resource wastage. The principle of allocating a quota depends on the demand of each AP, as shown in (9).

The computational complexity of obtaining a matching quota is $O(M\iota)$, in which ι is the computational complexity of the SARIMA model [60]. And there are *M* APs and *F* files. Each file can apply for *M* APs at most; thus, the matching process takes $O(FM)$ time complexity. Therefore, the time complexity of the algorithm is $O(M\iota + FM)$. The matching process is described as follows.

Step 1. Each content file proposes its favorite APs and removes this AP from its preference list (Lines 9-12).

Step 2. Each AP verifies all received proposals from the file (new proposals and those accepted in former iterations) and accepts the most preferred content files within the matching quota and rejects the remaining proposals (Lines 13-23).

Step 3. Return to *Step 1* to process all rejected files. The matching process terminates when all files are cached, or the quota of all APs is occupied.

Stability Analysis. Notably, the matching procedure is a mutual assignment between files and APs. The matching problem is a bilaterally stable matching, which is termed bilaterally stable [59] only when there are no blocking pairs (BP).

Definition 2: Matching Ψ is defined as stable if it is not blocked by any individual or pair.

To define stable matching, the parameters $F_i, i \in \psi^{\mathbf{F}}$ and $M_j, j \in \psi^{\mathbf{M}}$ are considered, such that $\Psi(M_j) \neq F_i$. The matching Ψ is blocked by an individual M_j or F_i, if M_j or F_i prefers a mismatch rather than the current match under Ψ. This implies that each M_j or F_i receives negative utility under Ψ. A matching Ψ is blocked by a pair (F_i, M_j) when the following conditions are satisfied: (1) The match is not blocked by individuals F_i and M_j; and (2) the utility value obtained by matching F_i and M_j is greater than that of the current match Ψ.

In the many-to-one matching process, each AP matches multiple files. Thus, we consider group stability, which comprises multiple stable pairs. A matching game is stable if all the matching pairs are stable.

Definition 3: The matching Ψ is defined as group stable if it is not blocked by any coalition.

To define group stability, let us assume that coalition $\mathcal{C}$ consists of a node and a group of objects that match. In many-to-one matching, a member of $\mathbf{M}$ is matched to a group of members of $\mathbf{F}$. The matching Ψ is blocked by a coalition $\mathcal{C}$ if there is another match Ψ' such that $\forall i, j \in \mathcal{C}$, where (1) $\Psi'(j) \in \mathcal{C}$; (2) $\mathcal{U}(\Psi'(j)) > \mathcal{U}(\Psi(j))$; (3) $\mathcal{U}(\Psi'(i)) > \mathcal{U}(\Psi(i))$; (4) if $u \in \Psi'(\psi^{\mathbf{M}})$ then $u \in \mathcal{C} \cup \Psi(\psi^{\mathbf{M}})$.

Condition (1) states that all $\mathbf{F}$ in $\mathcal{C}$ are matched to a member of $\mathbf{M}$ in $\mathcal{C}$. Conditions (2) and (3) demonstrate that all $\mathbf{F}$ and $\mathbf{M}$ in $\mathcal{C}$ prefer their current matches in Ψ' to their matches in Ψ. Condition (4) states that each member of $\mathbf{M}$ in $\mathcal{C}$ can be matched to a combination of a new member of $\mathbf{F}$ in $\mathcal{C}$ or a member of $\mathbf{F}$ that is matched under Ψ. If $\mathbf{F}$ and $\mathbf{M}$ in $\mathcal{C}$ find a match preferable to Ψ, then Ψ is blocked by some coalitions $\mathcal{C}$ of members of $\mathbf{F}$ and $\mathbf{M}$.

Theorem 1: The outcome convergence of the proposed matching algorithm is group stable.

To prove *Theorem 1*, it is convenient to present the following lemma.

Lemma 1: Matching is group stable if it is stable [66].

Proof: If a match is blocked by any individual or pair, it is clearly not group stable. This is because coalitions composed of these individuals or pairs are not stable.

*Proof of Theorem 1*According to Lemma 1, the proposed matching algorithm is proven to be stable. To prove that the proposed algorithm is not blocked by an individual, note that (1) each member of $\mathbf{F}$ and $\mathbf{M}$ will never receive a negative utility because the AP-level content popularity and traffic weight, which are the bases for establishing the preference list, are never negative. Additionally, (2) each member of $\mathbf{F}$ and $\mathbf{M}$ only matches objects that are in its preference list.

We now employ the contradiction method to prove that the match Ψ produced by the proposed algorithm is not blocked by any pair. Let us assume that $M_{i'}$ and $F_{j'}$ are blocking pairs. This implies that $\Psi(i') \neq j', \mathcal{U}_{F_{j'}}(i') > \mathcal{U}_{F_{j'}}(\Psi(j'))$ and $\mathcal{U}_{M_{i'}}(j') > \mathcal{U}_{M_{i'}}(\Psi(i'))$. Under the assumption $\mathcal{U}_{F_{j'}}(i') > \mathcal{U}_{F_{j'}}(\Psi(j'))$, content file $F_{j'}$ must have already been placed at AP $M_{i'}$ before it applies for a current partner $(\Psi(j'))$. However, $\Psi(i') \neq j'$ in matching results indicates that AP $M_{i'}$ prefers $\Psi(i')$ to $F_{j'}$ (*i.e.,* $\mathcal{U}_{M_{i'}}(j') < \mathcal{U}_{M_{i'}}(\Psi(i'))$ and rejects this proposal of $F_{j'}$. Therefore, the analysis result contradicts the assumption that the blocking pair formed by $M_{i'}$ and $F_{j'}$ does not exist. When the condition $\mathcal{U}_{M_{i'}}(j') > \mathcal{U}_{M_{i'}}(\Psi(i'))$ holds, the proof by contradiction based on the condition $\mathcal{U}_{F_{j'}}(i') > \mathcal{U}_{F_{j'}}(\Psi(j'))$ is similar to the previous description with minor modifications. Therefore, the proposed matching algorithm is stable. According to Lemma 1, this constitutes group-stable matching.

Theorem 1 states that any matched pair resulting from the proposed matching algorithm will not obtain higher utility than when partnering with any other pair.

Convergence Analysis. In a many-to-one matching game, contention exists among content files when the AP receives proposals from more than one content file. In the proposed matching algorithm, the matching rules indicate that the AP receives files that maximize the utility value of the AP. Because the content file only proposes to the AP on its preference list once, the proposal and rejection procedure ends when every file has matched an AP or has been rejected by all APs. Therefore, we can conclude that the proposed matching algorithm terminates after a finite number of iterations.

V. Experiment Setup

In this section, we describe the dataset, experiment settings, alternative cache strategies and performance metrics used in the test.

A. Experiment Settings

In the experiment, we used a high-performance server configured with an Intel Xeon(R) Silver 4210R CPU with 2.40 GHz*40, 64-GB DRAM, and 8-TB storage. We use the timestamp and user-AP associations provided by the Youku trace [46] to simulate the content request process. The trace contains 76 APs, 155,991 Youku video requests, 46,994 viewers, and 41,125 video clips of 25 types. Additionally, we implement a simulator for the edge caching framework, the intelligent gateway connecting with 76 APs. Based on the Youku trace, each AP covers a different number of end-users ranging from 1 to 10,296. We assume that the coverage of each AP is a circular area with a radius of 150 meters [70]. The Youku dataset contains the requested information of end-users connecting each AP without the geographical location of end-users. Thus, we assume that

TABLE II
PARAMETERS AND VALUES IN THE SIMULATION

Parameter	Values
Total number of APs	76
Total number of end-users	46,994
Total number of content files	41,125
Total number of requests	155,991
Size of each content file	1 MB
Radius of AP	150m
Channel bandwidth of AP	20 MHz
Total transmit power of AP	40 Watt
Path loss	36.8+36.7log(d), d[m]
Noise power	-95dBm
The delay between the AP and the gateway	100 ms

end-users are randomly distributed within the AP coverage. The path loss (dB) in the channel model is 36.8+36.7 log(d) [71], where *d* is the distance between an end-user and an AP in meter. We configure the channel bandwidth, total AP transmission power, and background noise powers of 20 MHz, 40 W, and -95 dBm, respectively [72]. The delay between the AP snd the gateway is set to 100 ms [73]. Besides, we set the size of all content files to be 1 MB. We list the parameters of the settings in Table II.

B. Compared Cache Placement Strategies

We employ six alternative cache strategies as follows.

GACD-N. The proposed GACD without popularity prediction of unknown domain content at APs.

FIFO [74]. The cache maintains a record of the first cached content. When the cache is full, the earliest cached content is replaced with a new one.

LRU [75]. The cache tracks the most recent requests of each cached content, and when the cache capacity is full, the least recent requested content is replaced with the new one.

LFU [76]. The cache tracks the number of requests for each cached content, and when the cache capacity is full, the least requested content is replaced with a new one.

LeCaR [77]. The cache manages two historical entries based on recency and frequency policies. The weight of each entry is updated using regret minimization, which determines the policy to be applied for cache eviction.

DRLCache [27]. The cache is built based on a deep reinforcement learning framework (that utilizes the Wolpertinger architecture) for the content-caching problem and is trained using a deep deterministic policy gradient.

To validate the efficiency of GACD, we compared it with two types of alternative caching strategies. First, FIFO, LRU, and LFU are traditional caching strategies that have been widely used in prior work. We prove that GACD is more suitable for latency-sensitive applications than these traditional caching strategies. Second, LeCaR is a machine learning-based caching algorithm that optimizes LRU and LFU. It relies on a probabilistic combination of the LRU and LFU strategies and attempts to learn the optimal combination of the two strategies using a regret minimization strategy. DRLCache is a cache strategy designed based on reinforcement learning for edge caching. Compared with the above caching strategies, the design of GACD is inspired by the observations from real-world trace analysis and optimizes both cache resource allocation and content placement to improve the caching performance, which has been validated by the experimental results.

Above algorithms are implemented and evaluated using the same edge-cache simulator to provide a fair comparison. In this study, we focus on the proactive cache content placement, but not cache content update [78]. In addition, we conduct 0-1 cache placement strategy instead of probabilistic cache placement [29], [31]. The strategies to optimize the cache content update and probabilistic cache placement are out of the scope of this study. Several existing works perform popularity prediction of old content files to determine cache placement strategies [30], [79]; however, we focus on the emergence of continuous new content files.

C. Performance Metrics

We use three performance metrics, including hit rate, average delay, and accuracy in the test. The high hit rate, low average delay, and high accuracy indicate that a cache strategy enables content files at APs to serve a large amount of end-users' requirements, and end-users achieve a high-quality experience. These metrics validate the benefits of our proposed GACD compared with alternative cache strategies.

Hit Rate (**HR**) is the ratio of the number of content files hit in the AP with respect to the total number of requests. This metric can be denoted by $\mathbf{HR} = (\sum_{m\in\mathbf{M}}^{M} N_m^h)/(\sum_{m\in\mathbf{M}}^{M}\sum_{k\in\mathbf{U}}^{K} n_k \cdot l_{m,k})$, where N_m^h is the number of satisfied user requests at AP AP_m.

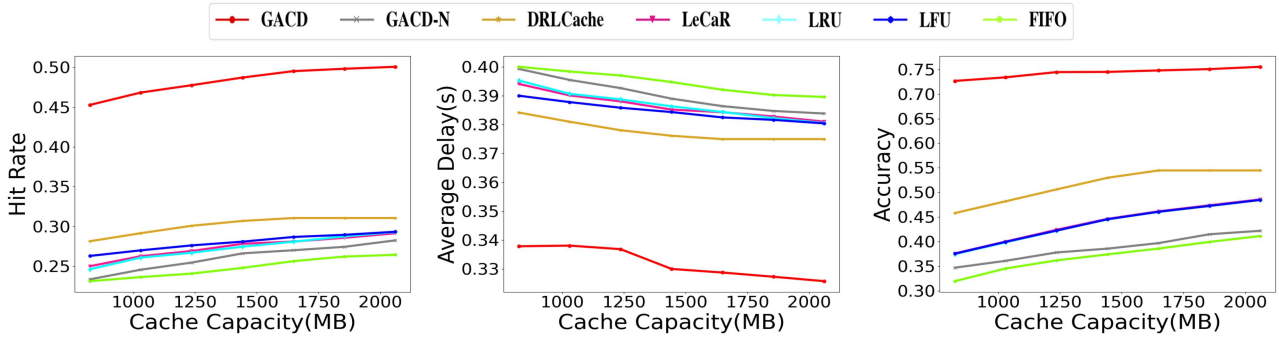
Average Delay (**AD**) means the average response delay from the cache at an AP to an end-user. This metric can be obtained by $\mathbf{AD} = \frac{1}{K}\sum_{k\in\mathbf{U}}^{K}\sum_{m\in\mathbf{M}}^{M}\sum_{i\in\mathbf{I}}^{F} r_{k,i} l_{m,k} d_{k,i}^m$, where $d_{k,i}^m$ is listed in (5). *Accuracy* (**A**) is the ratio of the number of content files right placed in the AP to the total content files. The accuracy can be calculated by $\mathbf{A} = \frac{TP+TN}{TP+TN+FP+FN}$. TP is the number of content files that are placed at APs and requested by end-users. TN is the number of content files that are placed at APs but not requested. FP represents the number of content files that are requested by end-users but not placed at APs. FN indicates the number of content files that are not requested by end-users and not placed at APs.

VI. PERFORMANCE EVALUATION

We study the performance of GACD under different configurations, including cache capacity, the number of APs, and the emergence of new content files. We conduct real-trace driven experiments to unveil the impact of the three parameters on cache hit rate, average delay, and accuracy.

A. GACD Performance Under Different Cache Capacity

We investigate the performance of cache strategies based on the cache capacity in APs. Fig. 10 shows the cache hit rate, average delay, and accuracy of the cache strategies. We set the available cache capacity range from 800 MB to 2000 MB, due



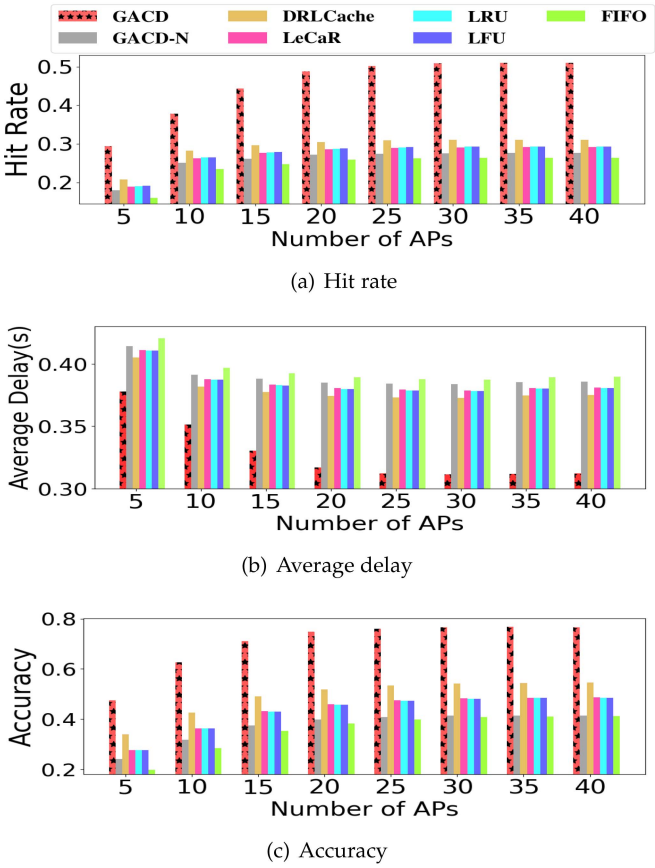
(a) Hit rate (b) Average delay (c) Accuracy

Fig. 10. Performance of cache strategies under different cache capacity.

to the assumed file size of 1 MB, which means that 800-2000 files can be stored in each selected APs. We select 40 APs to cache content files, and each AP obtains the matching quota allocation according to the underlying end-user requirements.[7] Fig. 10 shows that a large-capacity cache can place more content files, which improves the performance of all cache strategies. In Fig. 10(a), compared with GACD-N, DRLCache, LeCaR, LRU, LFU, and FIFO, GACD increases the hit ratio by up to 23.9%. However, the hit rate tends to be a stable value when the cache capacity exceeds 1800 files. The reason is that the cache capacity is large enough to store popular content files. In addition, a high hit rate reduces the average delay. GACD decreases the average delay by 15.4%, 13.14%, 14.52%, 14.59%, 14.38%, and 16.42%, respectively, see Fig. 10(b). This is because when cache capacity increases, end-users can get more requested content files from APs directly, reducing transmission delay. GACD improves the accuracy by 37.97%, 26.87%, 35.07%, 35.21%, 35.13%, and 40.72%, respectively, and the accuracy increases with the AP cache capacity (see Fig. 10(c)).

B. GACD Performance Under Different Numbers of APs

Fig. 11 shows the performance comparison of different cache strategies under different numbers of APs. The number of APs ranges from 5 to 40. Each AP has a fixed-size cache capacity (2000 files). Figs. 11(a)∼(c) present that GACD achieves higher performance than other strategies. GACD outperforms GACD-N, DRLCache, LeCaR, LRU, LFU, and FIFO by 23.32%, 19.88%, 21.80%, 21.65%, 21.61%, and 24.51%, respectively, in terms of the hit ratio when the number of APs is 30. However, the hit rate is stable when the number of AP reaches 40 from 25 because sufficient APs can serve all end-users requests. In Section II-B, some APs have few associations with end-users in the Youku dataset. If content files are cached on these APs, it is difficult to improve the hit rate in the cache. On the contrary, resources can be wasted owing to the cache cost. With 30 APs, GACD improves the accuracy by up to 35.31%, 22.46%, 28.39%, 28.5%, 28.49%, and 35.88%, compared with GACD-N, DRLCache, LeCaR, LRU, LFU, and FIFO, respectively.



(a) Hit rate

(b) Average delay

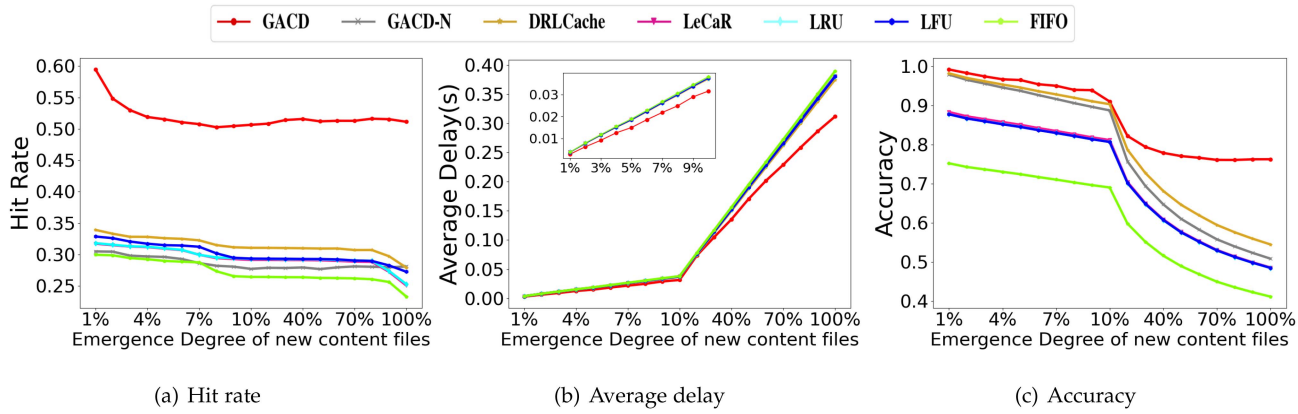
(c) Accuracy

Fig. 11. Performance of cache strategies under different number of APs.

C. Validity of New Content Popularity Prediction in Cache

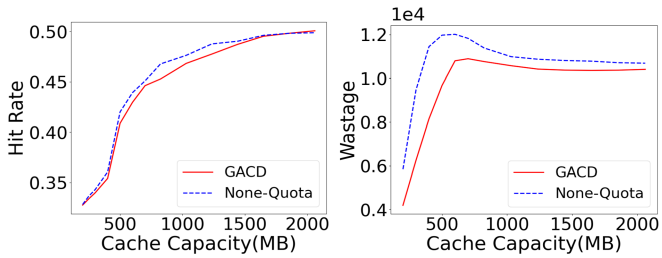
To validate the impact of the newly emerging content on cache performance, we suppose there are the N number of files including (40%× N) new files and (60%× N) old files. We employ S_{new} new files and $60\% \times N$ old files in a test. We define the metric *emergence degree* of new content files as $Edn = \frac{S_{new}}{40\% \times N}\%$. It must be noted that 100% means that all

[7]We selected the top 40 APs in Youku dataset with the highest total traffic consumption in a week to ensure that these APs in the experiment have different traffic consumption patterns and user access quantity.



(a) Hit rate (b) Average delay (c) Accuracy

Fig. 12. Performance of cache strategies under different emergence degree of new content files.



(a) Hit rate (b) Storage wastage

Fig. 13. Impact of dynamic quota allocation on hit rate and storage wastage.

new content files (40%$\times$ N) are used in this test. Fig. 12 presents the impact of different ratios of new content files on the hit rate, average delay, and accuracy. We select 40 APs and set their cache capacity to 2,000 MB. In Fig. 12, the performance of the cache strategies decreases when the number of new content files increases. GACD achieves an improvement of up to 29.5% in the hit rate compared with other strategies. As shown in Fig. 12(b), the average delay of all strategies increases when the percentage of new content increases. Moreover, GACD achieves the lowest average delay compared with other strategies. Compared with GACD-N, DRLCache, LeCaR, LRU, LFU, and FIFO, GACD improves the accuracy by up to 25.42%, 21.78%, 27.68%, 27.79%, 27.8%, and 35.13%, respectively, see Fig. 12(c). With more new content files, GACD outperforms alternative strategies in terms of the hit rate, average delay, and accuracy.

D. Validity of Dynamic Quota Allocation in the Cache

To validate dynamic quota allocation in the cache, we introduce a metric ('storage wastage'), which means the number of content files actively placed in the AP but not requested before the next cache content placement. In Fig. 13(a), the cache-hit rate of all cache strategies increases sharply with the cache capacity. Additionally, the two curves are close to each other. The performance of None-Quota is slightly better than GACD in terms of hit rate, with a maximum increase of 0.18%, which is a negligible improvement. No matter what the cache capacity is, GACD consistently outperforms the None-Quota strategy in saving cache resources. The storage-wastage gap between GACD and None-Quota strategy increases; subsequently, the gap decreases to a stable state, as shown in Fig. 13(b). When the cache capacity of each AP ranges from $[1000, 2000]$, the gap between GACD and None-Quota becomes smooth. GACD saves an average of 435 storage units compared to None-Quota. A storage unit stores a file. Thus, a dynamic quota allocation avoids the waste of storage at the edge cache, which effectively improves the cache resource utilization.

VII. Related Work

In this section, we present related works on intelligent gateway applications, distributed cache-placement policy, and popularity-based content cache strategies.

A. Intelligent Gateway

With the increment of computational capacity at the edge, the intelligent gateway becomes the bridging facility for the operations and function of sensor-cloud infrastructure. These gateways assist in managing data storage and analytics at the edge, integrate network protocols, and facilitate the secure flow of data between edge devices and the cloud [22].

Researchers are currently racing toward better evolution of IoT, while extracting valuable information from the vast amount of data generated in the IoT environment to provide better services. For instance, research [24] introduced an intelligent gateway, which has full knowledge and controls both the sensor network and the Internet. They can act as performance-enhancing proxies and intelligent caches to preserve the limited resources of the sensor network. The authors of [23] proposed a self-configurable gateway featuring real-time detection and configuration of smart things over the wireless networks. This gateway enables field data to be used for optimization, remote management, and preventive maintenance. The goal of the IoT gateway is to connect various sensor domain networks to the public communication network or the Internet, solving the problem of heterogeneity among various networks and enhancing the management of end nodes.

B. Distributed Cache-Placement Policy Optimization

Owing to the limited cache capacity, geographical locations, and content popularity in edge cache, many studies have been conducted on content-placement polices. It is essential to design an efficient content placement policy to maximize cache utilization by collaborating content management using limited resources.

Research [56] focused on the content placement problem in a wireless network formed by small cell access points and wireless users. The authors of [26] demonstrated a trade-off between the content diversity gain and the cooperative gain according to content placements. Several research [25], [27] have worked on using machine learning to enhance edge content placement. The previous works ignore temporal changes of content popularity, assuming that content popularity follows the Zipf distribution.

Nevertheless, content popularity usually varies over the time. A single popularity prediction model is difficult to adapt to all contents; thus, this model is ineffective in handling cache content placement. The authors of [15] proposed an accurate and adaptive popularity prediction model by training a simplified bidirectional long short-term memory network in cache-enabled fog radio access networks. In [80], a temporal convolutional network driven content popularity prediction model is developed to estimate the content popularity with high accuracy. In [28], the echo state networks were utilized to predict the content request distribution. These works mainly focused on the dynamic content popularity prediction instead of content placement strategies.

C. Popularity-Based Cache Placement Strategies

It is important to consider the content popularity in designing edge cache placement strategies. To utilize the resource at the edge, many pieces of prior work have been devoted to cache content placement based on content popularity.

Dynamic probabilistic caching strategies [29] were designed to handle content popularity that may change over time. In [31], online prediction and online learning methods are investigated for the prediction of content popularity. The authors of [30] propose a multi-armed bandit-based caching strategy to estimate an unknown but time-invariant content popularity distribution. Research [79] developed a hybrid multi-point process model to accurately predict the evolution of video popularity to optimize video caching.

However, these pieces of work ignored the fact that new content constantly flows into the Internet. These newly emerging content can be challenging to predict or model in advance [81]. A content-based approach [82] was applied to extract and condense the video features into a high-dimensional vector. They proposed a popularity prediction caching method to handle newly unpublished video files based on the content similarity between them and published files. A distributed caching policy based on the shot noise model was proposed in [83] to describe the popularity evolution of new content.

D. Efficient Resources Allocation Strategies

To manage limited resources at the edge, existing studies largely depended on accurate predictions of content popularity. The efficiency of placement decisions is also a factor that should be considered in the cache placement strategies. Effective cache placement strategies reduces the waste of resources.

The authors of [84] studied the dynamic provisioning of computing resources. Two allocation and pricing mechanisms were proposed to maximize social welfare in mobile edge computing (MEC) systems, where users have heterogeneous requests and compete for high-quality services. Research [85] jointly considered the quality of service and energy consumption in MEC-enabled service placement, an energy budget is set to avoid the waste of computation resources. A two-time-scale framework was proposed in [86] that jointly optimized service (code and data) placement and request scheduling.

Besides, maximizing utilization while reducing latency and energy consumption is still a key issue because the uneven distribution of user requests makes it difficult to determine which AP and what content should be cached. The authors of [14] studied a two-tier MEC system, which enables data caching and computing offloading policy to minimize the network cost at the user equipment (UE) side. In [87], the authors designed a privacy-preserving decentralized caching scheme to efficiently serve users in the low population density areas. Cache-enhanced computation offloading and content caching is traded off in [88] to obtain the optimal policy for allocating cache resources.

VIII. Conclusions and Future Work

In this study, we proposed an intelligent gateway-assisted cache deployment strategy for large-scale APs. The intelligent gateway undertakes data collection, cache management, and distribution to efficiently manage the limited cache resources of APs. We formulated the cache content problem as a many-to-one bidirectional matching problem with dynamic quota allocation, aiming to improve cache resources utilization and minimize the average content delivery latency. We designed an unknown domain files popularity prediction algorithm to predict the popularity of new content files and to mine users' preferences, and proposed a many-to-one file placement matching algorithm to achieve the dynamic matching quota adjustment and efficient content placement. We conducted extensive experiments to validate our GACD strategy in terms of the hit ratio, average delay, and accuracy. The results show that a dynamic quota allocation is important for effective utilization of cache resources.

In future, we will consider extending this work in the following directions. First, in order to further improve the hit rate, we will investigate strategies to fetch and replace content files during peak traffic. Second, we plan to locate and analyze more real-world traces and identify the scenarios where cache collaboration among APs can improve edge cache performance. Specifically, we will investigate the tradeoff between the collaborative cache mechanisms and the overlaps in user requests.

REFERENCES

[1] G. Forecast et al., "Cisco visual networking index: Global mobile data traffic forecast update, 2017–2022," *Update*, vol. 2017, 2019, Art. no. 2022.

[2] M. Ali, A. Anjum, O. Rana, A. R. Zamani, D. Balouek-Thomert, and M. Parashar, "RES: Real-time video stream analytics using edge enhanced clouds," *IEEE Trans. on Cloud Comput.*, vol. 10, no. 2, pp. 792–804, Second Quarter 2020.

[3] E. Ghabashneh and S. Rao, "Exploring the interplay between CDN caching and video streaming performance," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 516–525.

[4] T. Böttger, F. Cuadrado, G. Tyson, I. Castro, and S. Uhlig, "Open connect everywhere: A glimpse at the internet ecosystem through the lens of the Netflix CDN," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 48, no. 1, pp. 28–34, 2018.

[5] M. Calder, X. Fan, Z. Hu, E. Katz-Bassett, J. Heidemann, and R. Govindan, "Mapping the expansion of Google's serving infrastructure," in *Proc. Conf. Internet Meas. Conf.*, 2013, pp. 313–326.

[6] A.-J. Su, D. R. Choffnes, A. Kuzmanovic, and F. E. Bustamante, "Drafting behind Akamai (travelocity-based detouring)," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 36, no. 4, pp. 435–446, 2006.

[7] C. Huang, A. Wang, J. Li, and K. W. Ross, "Understanding hybrid CDN-P2P: Why limelight needs its own red swoosh," in *Proc. 18th Int. Workshop Netw. Operating Syst. Support Digit. Audio Video*, 2008, pp. 75–80.

[8] S. Yang, L. Jiao, R. Yahyapour, and J. Cao, "Online orchestration of collaborative caching for multi-bitrate videos in edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4207–4220, Dec. 2022.

[9] W. Hu, Y. Jin, Y. Wen, Z. Wang, and L. Sun, "Toward Wi-Fi AP-assisted content prefetching for an on-demand TV series: A learning-based approach," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 28, no. 7, pp. 1665–1676, Jul. 2018.

[10] Y. Guan, X. Zhang, and Z. Guo, "PrefCache: Edge cache admission with user preference learning for video content distribution," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 4, pp. 1618–1631, Apr. 2020.

[11] A. O. Al-Abbasi, V. Aggarwal, and M.-R. Ra, "Multi-tier caching analysis in CDN-based over-the-top video streaming systems," *IEEE/ACM Trans. Netw.*, vol. 27, no. 2, pp. 835–847, Apr. 2019.

[12] J. Zerwas, I. Poese, S. Schmid, and A. Blenk, "On the benefits of joint optimization of reconfigurable CDN-ISP infrastructure," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 1, pp. 158–173, Mar. 2022.

[13] K. Akpinar and K. A. Hua, "PPNET: Privacy protected CDN-ISP collaboration for QoS-aware multi-CDN adaptive video streaming," *ACM Trans. Multimedia Comput., Commun. Appl.*, vol. 16, no. 2, pp. 1–23, 2020.

[14] H. Feng, S. Guo, L. Yang, and Y. Yang, "Collaborative data caching and computation offloading for multi-service mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 70, no. 9, pp. 9408–9422, Sep. 2021.

[15] H. Feng, Y. Jiang, D. Niyato, F.-C. Zheng, and X. You, "Content popularity prediction via deep learning in cache-enabled fog radio access networks," in *Proc. IEEE Glob. Commun. Conf.*, 2019, pp. 1–6.

[16] U. Cisco, "Cisco annual internet report (2018–2023) white paper," Cisco, San Jose, CA, USA, Tech. Rep. C11-741490-01, 2020.

[17] F. Lyu et al., "LeaD: Large-scale edge cache deployment based on spatio-temporal wifi traffic statistics," *IEEE Trans. Mobile Comput.*, vol. 20, no. 8, pp. 2607–2623, Aug. 2020.

[18] Y. Liu, F. R. Yu, X. Li, H. Ji, and V. C. M. Leung, "Distributed resource allocation and computation offloading in fog and cloud networks with non-orthogonal multiple access," *IEEE Trans. Veh. Technol.*, vol. 67, no. 12, pp. 12137–12151, Dec. 2018.

[19] A. W. Services, "Intelligent gateways in microchip and amazon web services (AWS)," Jan. 25, 2012. Accessed: Apr. 2, 2022. [Online]. Available: https://www.microchip.com/en-us/solutions/internet-of-things/amazon-web-services/intelligent-gateways#

[20] L. Four-Faith Communication Technology Co, "Intelligent Gateway FG100," Feb. 14, 2019. Accessed: Apr. 2, 2022. [Online]. Available: https://en.four-faith.com/Intelligent-Gateway/

[21] I. Motorola Solutions, "MC-Edge intelligent gateway," Jan. 1, 2021. Accessed: Feb., 2022. [Online]. Available: https://www.motorolasolutions.com/en_xp/products/industrial-internet-of-things/scada-systems/mc-edge.html#tabproductinfo

[22] V. Hayyolalam, M. Aloqaily Özkasap, and M. Guizani, "Edge-assisted solutions for IoT-based connected healthcare systems: A literature review," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9419–9443, Jun. 2022.

[23] B. Kang, D. Kim, and H. Choo, "Internet of everything: A large-scale autonomic IoT gateway," *IEEE Trans. Multi-Scale Comput. Syst.*, vol. 3, no. 3, pp. 206–214, Third Quarter 2017.

[24] D. Bimschas, H. Hellbrück, R. Mietz, D. Pfisterer, K. Römer, and T. Teubler, "Middleware for smart gateways connecting sensornets to the internet," in *Proc. 5th Int. Workshop Middleware Tools, Serv. Run-Time Support Sensor Netw.*, 2010, pp. 8–14.

[25] X. Cao, J. Zhang, and H. V. Poor, "An optimal auction mechanism for mobile edge caching," in *Proc. IEEE 38th Int. Conf. Distrib. Comput. Syst.*, 2018, pp. 388–399.

[26] S. H. Chae, T. Q. Quek, and W. Choi, "Content placement for wireless cooperative caching helpers: A tradeoff between cooperative gain and content diversity gain," *IEEE Trans. Wireless Commun.*, vol. 16, no. 10, pp. 6795–6807, Oct. 2017.

[27] C. Zhong, M. C. Gursoy, and S. Velipasalar, "Deep reinforcement learning-based edge caching in wireless networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 1, pp. 48–61, Mar. 2020.

[28] M. Chen, W. Saad, C. Yin, and M. Debbah, "Echo state networks for proactive caching in cloud-based radio access networks with mobile users," *IEEE Trans. Wireless Commun.*, vol. 16, no. 6, pp. 3520–3535, Jun. 2017.

[29] J. Gao, S. Zhang, L. Zhao, and X. Shen, "The design of dynamic probabilistic caching with time-varying content popularity," *IEEE Trans. Mobile Comput.*, vol. 20, no. 4, pp. 1672–1684, Apr. 2021.

[30] J. Song, M. Sheng, T. Q. Quek, C. Xu, and X. Wang, "Learning-based content caching and sharing for wireless networks," *IEEE Trans. Commun.*, vol. 65, no. 10, pp. 4309–4324, Oct. 2017.

[31] N. Garg, M. Sellathurai, V. Bhatia, B. Bharath, and T. Ratnarajah, "Online content popularity prediction and learning in wireless edge caching," *IEEE Trans. Commun.*, vol. 68, no. 2, pp. 1087–1100, Feb. 2020.

[32] G. Zhang, T. Q. Quek, M. Kountouris, A. Huang, and H. Shan, "Fundamentals of heterogeneous backhaul design analysis and optimization," *IEEE Trans. Commun.*, vol. 64, no. 2, pp. 876–889, Feb. 2016.

[33] G. Zhang, T. Q. Quek, A. Huang, M. Kountouris, and H. Shan, "Delay modeling for heterogeneous backhaul technologies," in *Proc. IEEE 82nd Veh. Technol. Conf.*, 2015, pp. 1–6.

[34] N. Alliance, "5G white paper," *Next generation mobile networks, white paper*, vol. 1, no. 2015, 2015.

[35] F. Qamar, M. Hindia, K. Dimyati, K. A. Noordin, and I. S. Amiri, "Interference management issues for the future 5G network: A review," *Telecommunication Syst.*, vol. 71, no. 4, pp. 627–643, 2019.

[36] A. A. Khan, M. Abolhasan, and W. Ni, "5G next generation VANETs using SDN and fog computing framework," in *Proc. IEEE 15th Annu. Consum. Commun. Netw. Conf.*, 2018, pp. 1–6.

[37] A. Narayanan et al., "A first look at commercial 5G performance on smartphones," in *Proc. Web Conf.*, 2020, pp. 894–905.

[38] Y. Zhao, J. Zhao, W. Zhai, S. Sun, D. Niyato, and K.-Y. Lam, "A survey of 6G wireless communications: Emerging technologies," in *Proc. Future Inf. Commun. Conf.*, Springer, 2021, pp. 150–170.

[39] K. Sui et al., "Characterizing and improving WiFi latency in large-scale operational networks," in *Proc. 14th Annu. Int. Conf. Mobile Syst., Appl., Serv.*, 2016, pp. 347–360.

[40] X. Wang, Y. Ji, J. Zhang, L. Bai, and M. Zhang, "Low-latency oriented network planning for MEC-enabled WDM-PON based fiber-wireless access networks," *IEEE Access*, vol. 7, pp. 183383–183395, 2019.

[41] M. Chen, M. Mozaffari, W. Saad, C. Yin, M. Debbah, and C. S. Hong, "Caching in the sky: Proactive deployment of cache-enabled unmanned aerial vehicles for optimized quality-of-experience," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 5, pp. 1046–1061, May 2017.

[42] D. Jiang and Y. Cui, "Analysis and optimization of caching and multicasting for multi-quality videos in large-scale wireless networks," *IEEE Trans. Commun.*, vol. 67, no. 7, pp. 4913–4927, Jul. 2019.

[43] C. Li, J. Liu, and S. Ouyang, "Where do you watch? a spatial analysis of online video service in mobile network," in *Proc. IEEE 19th Int. Symp. Wireless Pers. Multimedia Commun.*, 2016, pp. 108–114.

[44] C. Li and J. Liu, "Large-scale characterization of comprehensive online video service in mobile network," in *Proc. IEEE Int. Conf. Commun.*, 2016, pp. 1–7.

[45] C. Li, J. Liu, and S. Ouyang, "Analysis and prediction of content popularity for online video service: A Youku case study," *China Commun.*, vol. 13, no. 12, pp. 216–233, 2016.

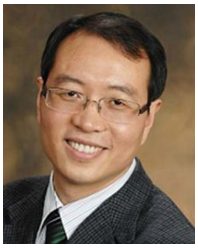
[46] S. Ouyang, C. Li, and X. Li, "Characterizing the service usage of online video sharing system: Uploading vs. playback," in *Proc. IEEE 19th Int. Symp. Wireless Pers. Multimedia Commun.*, 2016, pp. 280–286.

[47] C. Li, J. Liu, and S. Ouyang, "Characterizing and predicting the popularity of online videos," *IEEE Access*, vol. 4, pp. 1630–1641, 2016.

[48] S. Yan, L. Qi, Y. Zhou, M. Peng, and G. S. Rahman, "Joint user access mode selection and content popularity prediction in non-orthogonal multiple access-based F-RANs," *IEEE Trans. Commun.*, vol. 68, no. 1, pp. 654–666, Jan. 2019.
[49] Y. Zhou, S. Yan, and M. Peng, "Content placement with unknown popularity in fog radio access networks," in *Proc. IEEE Int. Conf. Ind. Internet*, 2019, pp. 361–367.
[50] W. Nie, F.-C. Zheng, X. Wang, W. Zhang, and S. Jin, "User-centric cross-tier base station clustering and cooperation in heterogeneous networks: Rate improvement and energy saving," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 5, pp. 1192–1206, May 2016.
[51] S. Mosleh, Q. Fan, L. Liu, J. D. Ashdown, E. Perrins, and K. Turck, "Content-aware user association and multi-user MIMO beamforming over mobile edge caching," 2019, *arXiv:1906.11318*.
[52] Y. Li, H. Ma, L. Wang, S. Mao, and G. Wang, "Optimized content caching and user association for edge computing in densely deployed heterogeneous networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 2130–2142, Jun. 2022.
[53] M. De Mari, E. C. Strinati, M. Debbah, and T. Q. Quek, "Joint stochastic geometry and mean field game optimization for energy-efficient proactive scheduling in ultra dense networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 3, no. 4, pp. 766–781, Dec. 2017.
[54] Y. Jiang, Y. Hu, M. Bennis, F.-C. Zheng, and X. You, "A mean field game-based distributed edge caching in fog radio access networks," *IEEE Trans. Commun.*, vol. 68, no. 3, pp. 1567–1580, Mar. 2020.
[55] D. C. Chen, T. Q. S. Quek, and M. Kountouris, "Backhauling in heterogeneous cellular networks: Modeling and tradeoffs," *IEEE Trans. Wireless Commun.*, vol. 14, no. 6, pp. 3194–3206, Jun. 2015.
[56] K. Shanmugam, N. Golrezaei, A. G. Dimakis, A. F. Molisch, and G. Caire, "FemtoCaching: Wireless content delivery through distributed caching helpers," *IEEE Trans. Inf. Theory*, vol. 59, no. 12, pp. 8402–8413, Dec. 2013.
[57] C. Shi, Y. Li, J. Zhang, Y. Sun, and P. S. Yu, "A survey of heterogeneous information network analysis," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 1, pp. 17–37, Jan. 2017.
[58] C. Shi, B. Hu, W. X. Zhao, and S. Y. Philip, "Heterogeneous information network embedding for recommendation," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 2, pp. 357–370, Feb. 2019.
[59] Y. Gu, W. Saad, M. Bennis, M. Debbah, and Z. Han, "Matching theory for future wireless networks: Fundamentals and applications," *IEEE Commun. Mag.*, vol. 53, no. 5, pp. 52–59, May 2015.
[60] S. P. Sone, J. J. Lehtomäki, and Z. Khan, "Wireless traffic usage forecasting using real enterprise network data: Analysis and methods," *IEEE Open J. Commun. Soc.*, vol. 1, pp. 777–797, 2020.
[61] J. An, H. Yang, X. Gui, W. Zhang, R. Gui, and J. Kang, "TCNS: Node selection with privacy protection in crowdsensing based on twice consensuses of blockchain," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 3, pp. 1255–1267, Sep. 2019.
[62] D. C. Nguyen et al., "Federated learning meets blockchain in edge computing: Opportunities and challenges," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 12806–12825, Aug. 2021.
[63] Y. Sun, J. Han, X. Yan, P. S. Yu, and T. Wu, "PathSim: Meta path-based top-K similarity search in heterogeneous information networks," *Proc. VLDB Endowment*, vol. 4, no. 11, pp. 992–1003, 2011.
[64] Y. Dong, N. V. Chawla, and A. Swami, "Metapath2vec: Scalable representation learning for heterogeneous networks," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 135–144.
[65] A. Mnih and R. R. Salakhutdinov, "Probabilistic matrix factorization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2008, pp. 1257–1264.
[66] A. E. Roth and M. Sotomayor, "Two-sided matching," *Handbook Game Theory Econ. Appl.*, vol. 1, pp. 485–541, 1992.
[67] T. Liu et al., "Distributed file allocation using matching game in mobile fog-caching service network," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2018, pp. 499–504.
[68] J. Li et al., "On social-aware content caching for D2D-enabled cellular networks with matching theory," *IEEE Internet Things J.*, vol. 6, no. 1, pp. 297–310, Feb. 2019.
[69] H. Wu et al., "Delay-minimized edge caching in heterogeneous vehicular networks: A matching-based approach," *IEEE Trans. Wireless Commun.*, vol. 19, no. 10, pp. 6409–6424, Oct. 2020.
[70] Z. Yang et al., "Cache placement in two-tier hetnets with limited storage capacity: Cache or buffer?," *IEEE Trans. Commun.*, vol. 66, no. 11, pp. 5415–5429, Nov. 2018.
[71] E. Dinc and O. B. Akan, "Channel model for the surface ducts: Large-scale path-loss, delay spread, and AOA," *IEEE Trans. Antennas Propag.*, vol. 63, no. 6, pp. 2728–2738, Jun. 2015.
[72] X. Wang, R. Li, C. Wang, X. Li, T. Taleb, and V. C. Leung, "Attention-weighted federated deep reinforcement learning for device-to-device assisted heterogeneous collaborative edge caching," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 154–169, Jan. 2020.
[73] V. Navda, A. Kashyap, and S. R. Das, "Design and evaluation of imesh: An infrastructure-mode wireless mesh network," in *Proc. IEEE 6th Int. Symp. World Wireless Mobile Multimedia Netw.*, 2005, pp. 164–170.
[74] D. Rossi and G. Rossini, "Caching performance of content centric networks under multi-path routing (and more)," *Relatório Técnico, Telecom ParisTech*, vol. 2011, pp. 1–6, 2011.
[75] M. Ahmed, S. Traverso, P. Giaccone, E. Leonardi, and S. Niccolini, "Analyzing the performance of LRU caches under non-stationary traffic patterns," 2013, *arXiv:1301.4909*.
[76] A. Jaleel, K. B. Theobald, S. C. Steely Jr, and J. Emer, "High performance cache replacement using re-reference interval prediction (RRIP)," *ACM SIGARCH Comput. Archit. News*, vol. 38, no. 3, pp. 60–71, 2010.
[77] G. Vietri et al., "Driving cache replacement with ML-based LeCaR," in *Proc. 10th USENIX Workshop Hot Topics Storage File Syst.*, 2018.
[78] M. Ma and V. W. Wong, "Age of information driven cache content update scheduling for dynamic contents in heterogeneous networks," *IEEE Trans. Wireless Commun.*, vol. 19, no. 12, pp. 8427–8441, Dec. 2020.
[79] X. Zhang et al., "Optimizing video caching at the edge: A hybrid multi-point process approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 10, pp. 2597–2611, Oct. 2022.
[80] X. Zhang, Z. Qi, G. Min, W. Miao, Q. Fan, and Z. Ma, "Cooperative edge caching based on temporal convolutional networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 9, pp. 2093–2105, Sep. 2022.
[81] T. Zhao, I.-H. Hou, S. Wang, and K. Chan, "Red/LeD: An asymptotically optimal and scalable online algorithm for service caching at the edge," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 8, pp. 1857–1870, Aug. 2018.
[82] K. N. Doan, T. Van Nguyen, T. Q. Quek, and H. Shin, "Content-aware proactive caching for backhaul offloading in cellular network," *IEEE Trans. Wireless Commun.*, vol. 17, no. 5, pp. 3128–3140, May 2018.
[83] J. Ji, K. Zhu, R. Wang, B. Chen, and C. Dai, "Energy efficient caching in backhaul-aware cellular networks with dynamic content popularity," *Wireless Commun. Mobile Comput.*, vol. 2018, pp. 5612–5648, 2018.
[84] T. Bahreini, H. Badri, and D. Grosu, "Mechanisms for resource allocation and pricing in mobile edge computing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 667–682, Mar. 2022.
[85] H. Badri, T. Bahreini, D. Grosu, and K. Yang, "Energy-aware application placement in mobile edge computing: A stochastic optimization approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 4, pp. 909–922, Apr. 2020.
[86] V. Farhadi et al., "Service placement and request scheduling for data-intensive applications in edge clouds," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 779–792, Apr. 2021.
[87] Y. Zeng, Y. Huang, Z. Liu, J. Liu, and Y. Yang, "Privacy-preserving decentralized edge caching in 5G networks," in *Proc. IEEE 14th Int. Conf. Cloud Comput.*, 2021, pp. 189–199.
[88] Y. Dong, S. Guo, Q. Wang, S. Yu, and Y. Yang, "Content caching-enhanced computation offloading in mobile edge service networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 1, pp. 872–886, Jan. 2022.

Hui Sun (Member, IEEE) received the PhD degree from Huazhong University Science and Technology, in 2014. He is an associate professor of computer science with Anhui University. His research interests include computer systems and edge computing.

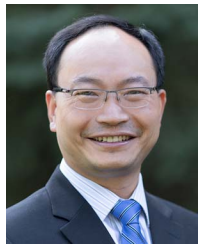
Yiru Chen is currently working toward the MS degree with Anhui University. Her main research interests include edge computing and computer systems.



Kewei Sha (Senior Member, IEEE) received the PhD degree in computer science from Wayne State University, in 2008. He is an associate professor of computer science and the associate director with the Cyber Security Institute, University of Houston-Clear Lake (UHCL). His research interests include Internet of Things, cyber-physical systems, edge computing, security and privacy, and data management and analytics. His research has been supported by NSF, NASA, UHCL and OCU. He has served as an associate editor of *IEEE IoT Journal*, *Elsevier Smart Health* and *Springer Computing*, a guest editor with several prestigious international journals, and an organizing committee member of many conferences. He is a senior member of ACM.

Shaoyuan Huang is currently working toward the PhD degree with Tianjing University. His main research interests include edge computing, Internet of Things, network.

Xiaofei Wang (Senior Member, IEEE) received the master's and doctor degrees from Seoul National University, in 2006 and 2013. He is currently a professor with the Tianjin Key Laboratory of Advanced Networking, School of Computer Science and Technology, Tianjin University, China. He was a post-doctoral fellow with The University of British Columbia from 2014 to 2016. Focusing on the research of social-aware cloud computing, cooperative cell caching, and mobile traffic offloading, he has authored more than 100 technical papers in the *IEEE Journal on Selected Areas in Communications*, the *IEEE Transactions on Wireless Communications*, the *IEEE Wireless Communications*, the *IEEE Communications*, the *IEEE Communications*, the IEEE INFOCOM, and the IEEE SECON. He was a recipient of the National Thousand Talents Plan (Youth) of China.



Weisong Shi (Fellow, IEEE) received the BS degree from Xidian University, in 1995, and the PhD degree from the Chinese Academy of Sciences, in 2000, both in computer engineering. He is a professor and chair with the Department of Computer and Information Sciences, the University of Delaware His research interests include edge computing, computer systems, and wireless health. He is a recipient of the National Outstanding PhD dissertation award of China and the NSF CAREER award. He is a fellow of ACM distinguished scientist.